University of Calabria

Department of Mathematics and Computer Science

Master's Degree Course in Computer Science

Master's Thesis

Lazy Declarative Heuristics in Answer Set Programming: Design and Evaluation of a clingo Propagator

Supervisor at the University of Calabria:
Prof. Carmine Dodaro

Supervisors at the University of Potsdam:
Prof. Dr. Torsten Schaub
Dr. Javier Romero Dávila

Candidate:
Pierpaolo Spadafora

Student ID 263722

Academic Year 2025/2026

Contents

Abstract

In the area of Answer Set Programming (ASP), ground-and-solve systems such as clingo require the instantiation of first-order rules before search begins. Although declarative domain-specific heuristics provide valuable search guidance, dynamic heuristics that depend on runtime search state (such as dynamic aggregates) suffer from combinatorial explosion during grounding. Conversely, lazy-grounding systems evaluate heuristics dynamically over partial assignments, but incur substantial search overhead because grounding and solving are interleaved throughout search. This thesis introduces a lazy declarative heuristic framework for clingo using its propagator interface, combining dynamic heuristic evaluation with the efficiency of ground-and-solve systems.

Two execution backends are provided: a compiled C++ backend with incremental aggregate state maintenance, and an embedded SWI-Prolog engine with relational query evaluation over a synchronized solver trail. Our methodology decouples heuristic representation (ground vs. lazy) from interpretation semantics (clingo-like vs. Alpha-like) through order-preserving weight flattening and the appropriate evaluation of default negation on unassigned atoms. We demonstrate that this semantic alignment is essential for dynamic heuristics to correctly steer the search. Benchmark results on three hard combinatorial problem families (the Balanced Sum Problem, the Partner Units Problem, and the House Reconfiguration Problem) show that the lazy propagator avoids grounding bottlenecks and drastically reduces memory consumption while preserving optimal search guidance. Compared to the lazy-grounding solver Alpha, our framework achieves the performance of ground-and-solve systems while maintaining the low memory cost of lazy-grounding solvers evaluating dynamic aggregates in heuristics.

Chapter 1

Introduction and Background

1.1 The Cost of a Dynamic Heuristic

Answer Set Programming (ASP) is a declarative approach to combinatorial search. Instead of describing an algorithm step by step, the modeler describes the properties that an admissible solution must satisfy, and a solver enumerates the assignments that satisfy them [1, 2, 3].

ASP solvers are essentially highly optimized backtracking, or more appropriately *backjumping*, engines. Modern ASP solving is dominated by two main paradigms: ground-and-solve and lazy grounding.

In ground-and-solve systems, such as clingo, first-order rules are instantiated over the problem domain before search starts, including the domain-specific heuristics that direct the solver's search. This approach can be severely memory-intensive, with grounding bottlenecks manifesting in two main scenarios: as problem instances scale, and whenever domain-specific heuristics depend on dynamic search state, for instance through an aggregate over the atoms that are true *at that point of the search*. Because grounding precedes search, the grounder must enumerate every possible value the aggregate could take, and a single heuristic ends up producing a number of ground objects that is combinatorial in the size of the instance. In contrast, lazy-grounding systems interleave grounding and solving. While this yields significantly higher memory efficiency, allowing a further reach on memory-intensive problems, it comes at the cost of a significantly slower search.

Depending on its underlying architecture, an ASP solver can be seen as an engine that compiles or interprets a very convenient and elegant declarative language [4] to search for a solution. The convenience of that language is paid for in control: the modeler states *what* an admissible solution is, but not *how* the engine should look for one. A pure backtracking approach behaves as an uninformed search, and on problems whose structure is known by construction, guiding the solver with that knowledge is extremely helpful.

Domain-specific heuristics are the means by which the modeler feeds that knowledge into the search. Syntactically they are declarative annotations that the grounder instantiates like ordinary rules, and their purpose is to tell the solver which atom to decide next, and with which polarity; their impact on solving is well documented [5, 6]. They are, however, part of the program, and clingo instantiates the program before search starts. A heuristic is therefore expanded, in full, before the first decision is taken. That expansion can be relatively harmless as long as the heuristic depends only on the input. It stops being harmless as soon as the heuristic depends on *how the search is going*, which is precisely when a heuristic is most useful. Consider the directive used throughout this thesis to guide the Balanced Sum Problem (BSP), the problem of splitting $\{1, \ldots, n\}$ into two sets `b/1` and `c/1` of nearly equal sum:

```
#heuristic b(X) : x(X), not c(X), S = #sum{Y : c(Y)}, W = ((n+1)*S)+X. [W, true]
```

The rule reads: prefer to place the element X into b, the more strongly the larger the sum S of what is already in c. The aggregate value S appears *inside the weight*, and the atoms `c(Y)` it sums over are guessed during search rather than given as input facts. Since the grounder cannot know what S will be at decision time, it must enumerate every potential value of the sum ($S \in \{0, \ldots, \frac{n(n+1)}{2}\}$) and instantiate a separate ground directive for every pair (X, S). This yields $\Theta(n^3)$ ground objects (over one million (1,010,200) directives at $n = 100$) for the symmetric rules of $b/1$ and $c/1$. All of them instantiate the same single rule the modeler wrote, and all but the one matching the current partial sum are inert at any given decision. A domain-specific heuristic introduced to help the engine solve a problem quicker ends up putting the answer out of reach.

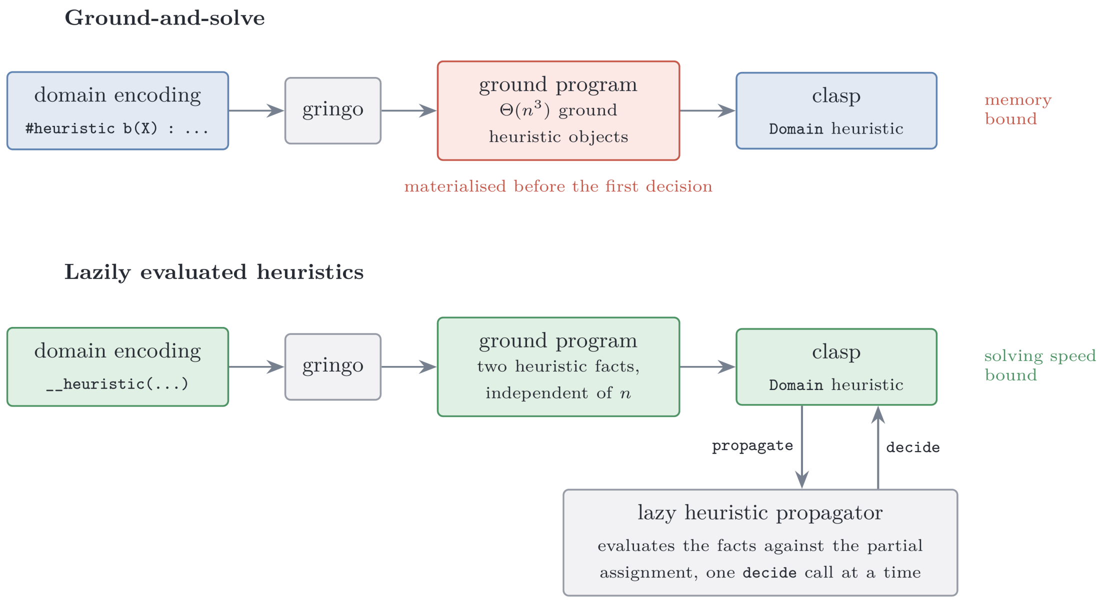


Figure 1.1: The same heuristic under the two approaches compared in this thesis.

The alternative is to leave the heuristic symbolic and to evaluate it while the search runs, when the partial assignment that S refers to actually exists. This is the natural approach in lazy-grounding solvers such as Alpha [7, 8, 9].

1.2 Answer Set Programming

A *normal rule* in ASP has the form

$$a \leftarrow b_1, \ldots, b_m, \ \textit{not } c_1, \ldots, \textit{not } c_n,$$

where a, the b_i and the c_j are atoms: the head a is derived whenever every positive body atom b_i is derivable and no negative body atom c_j holds. The negation *not* is *default negation*: *not* c holds in the absence of a derivation for c. Because a rule read this way can justify itself, the meaning of a program is not given by the rules alone but through the notion of a stable model. Fix a set of atoms X, the candidate solution, and a ground program P. The *reduct* P^X is obtained by deleting every rule whose negative body intersects X, and by deleting the negative literals from the rules that survive. The reduct is a positive program, so it has a unique least model, and X is a *stable model* (or *answer set*) of P exactly when it coincides with that least model [1]. On the two-rule program

$$\begin{Bmatrix} a \leftarrow \textit{not } b \\ b \leftarrow \textit{not } a \end{Bmatrix}$$

the candidate $X = \{a\}$ gives the reduct $\{a \leftarrow\}$, whose least model is $\{a\}$: stable. The candidate $\{a, b\}$ deletes both rules, leaving the least model $\emptyset$: not stable. A stable model is then a set of atoms both closed under the rules and justified by them, in which every atom has a non-circular derivation that survives the model's own assumptions about what is false.

In the *guess-and-check* idiom a choice construct opens the solution space, integrity constraints prune the candidates that violate the specification, and the answer sets of the program correspond one-to-one to the solutions of the problem. ASP syntax extends normal rules with first-order variables, arithmetic, conditional literals, and three constructs this thesis uses throughout. A *choice rule* such as `{ b(X) } :- x(X).` leaves the truth values of its head atoms unconstrained, making these *choice atoms* the explicit decision points of the solver. An *integrity constraint* `:- Body.` is a rule with empty head and forbids every model that satisfies *Body*. An *aggregate atom* such as `#sum`, `#count`, `#min` or `#max` condenses a set of weighted contributions into a single comparable value [3]. Aggregates are central to this work: the heuristics under study rank decisions by expressions such as "the current sum of the elements already placed in a set", and how and *when* such an aggregate is evaluated is exactly what differentiates the systems compared here.

1.3 Ground-and-Solve: Gringo, Clasp, and Conflict-Driven Search

clingo [10] couples the grounder Gringo with the solver Clasp into a single system. The grounder instantiates the first-order program over its Herbrand universe, producing a variable-free program. To mitigate combinatorial explosion, Gringo restricts this expansion to rule instances whose positive body is potentially derivable;

as a result, the ground program is generally much smaller than a full instantiation, though it must still be materialized in full before search starts. The solver Clasp [11] then searches for the stable models of the ground program using *conflict-driven nogood learning* (CDNL), an algorithm that learns from conflict-inducing choices. The solver maintains a *partial assignment*, a consistent set of literals over the program atoms; it alternates *unit propagation* steps with *decision* steps, and when propagation derives a contradiction, it analyzes the conflict, learns a nogood built around the *First Unique Implication Point* (1UIP), and *backjumps* to the decision level at which that nogood becomes unit. Which atom to decide on, and with which polarity, is delegated to a *decision heuristic*; Clasp's default is activity-based, in the family introduced by [12], and favors atoms that appear in recent conflicts.

This approach entails that every atom and every rule Clasp will ever see must exist before the first decision is made and also that the size of the ground program is the dominant memory cost of the pipeline. The *grounding bottleneck* is a well-known limitation of ground-and-solve systems [8]; Chapter 4 measures one concrete manifestation of it, namely heuristic directives whose weight depends on an aggregate value.

1.4 Domain-Specific Heuristics in clingo

Clasp allows the modeler to steer its decision heuristic using domain-specific knowledge through the `#heuristic` directive [5]:

```
#heuristic A : Body. [W@P, M]
```

Here, A is the target domain atom whose decision choice is steered according to the modifier M (e.g., setting its sign or priority level). The heuristic applies only if the rule *Body* is satisfied. If multiple heuristic directives target the same atom A, clingo selects the one with the highest priority P to determine the atom's modifier and weight. Across distinct target atoms, Clasp then prioritizes candidates with higher weight W. In contrast, Alpha treats (P, W) as a global lexicographic rank over all atoms (Section 1.5).

1.5 Lazy Grounding, Alpha, and Heuristics over Partial Assignments

Alpha is the reference state-of-the-art lazy-grounding solver for this work.

Comploi-Taupe et al. extended Alpha with declaratively specified domain-specific heuristics evaluated against the *partial* assignment maintained during solving [8]. Its semantics differs from clingo's on two main points.

Specifically, Alpha evaluates negated body literals against unassigned atoms and prioritizes P over weight W in the tuple $(W@P)$, always preferring higher priority values. Only when two atoms, even different, have the same priority are they compared by weight. A complete formalization of these semantic differences and their integration into our experimental design is provided in Chapter 2.

1.6 Extending clingo: Propagators and the Heuristic Interface

Standard clingo does not evaluate heuristic conditions dynamically at search time. However, it provides an extension mechanism: *theory propagators*, user-defined components registered with the solver that participate directly in the search loop [10]. This thesis leverages that interface to provide search guidance computed dynamically on the trail rather than fully materialized before search.

A propagator has four callbacks. During `init`, it inspects the symbolic atoms of the ground program, maps the atoms it cares about to solver literals, and registers *watches* on them. During search, `propagate` is invoked when a watched literal becomes true, `undo` when backtracking retracts it, and `check` on total assignments. The propagator is invoked by Clasp only when a watched literal changes state.

The `clingo::Heuristic` interface inherits from Propagator and extends it with a `decide` callback, invoked at each decision step *before* the solver's built-in heuristic. The Heuristic object receives the current assignment of the solver and either returns a signed literal, which becomes the next decision, or returns 0, in which case Clasp falls back to its default activity-based heuristic (VSIDS [12]). This is the mechanism the rest of the thesis builds on. It keeps the declarative heuristic descriptions in memory, evaluates them incrementally against the trail, and answers each `decide` call with the best applicable target.

1.7 Objective and Contributions

This work asks whether the same effect of lazily evaluating heuristics can be obtained inside clingo without giving up its ground-and-solve pipeline and speed advantage, that is, whether a ground-and-solve engine can be equipped with lazily evaluated heuristics (Figure 1.1) and whether the overhead of dynamic evaluation still allows the engine to outperform a state-of-the-art lazy-grounding solver.

The thesis makes the following contributions.

1. **A lazy heuristic evaluator for clingo:** A propagator, registered through the public `clingo::Heuristic` interface, that keeps a declarative heuristic in memory as a compact description and evaluates it at every decision against the current partial assignment.

2. **Two propagator backends:** A compiled C++ backend, which materializes one candidate per ground target and maintains incremental aggregate states, and an in-process SWI-Prolog backend, which evaluates the heuristic as an ordinary logic program against a mirrored image of the solver trail.

3. **Semantics and ablation study:** Ground-and-Solve, Lazy, clingo semantics and Alpha semantics combined give four variants of every encoding plus a heuristic-free baseline, so that no measured difference has to be attributed to two causes at once.

4. **Weight translation from Alpha's syntax:** Alpha's priority levels are global, while clingo's are local to a single atom. Section 2.1.3 presents the transformation

that folds a level into the weight, the conditions under which it preserves the intended order and the weight field limit imposed by clingo.

5. **Trade-off analysis of the lazy approach.** By collecting per-decision metrics in both backends, we evaluate the exact computational overhead of lazy grounding against its savings in program size.

The evaluation covers three problem families, the Balanced Sum Problem, the Partner Units Problem, and the House Reconfiguration Problem, on both backends, with a correctness harness that checks that no variant changes the answer sets. The implementation is available at[1].

[1] `https://github.com/pierspad/clingo-lazy-heuristic-evaluator`

Chapter 2

Methodology

2.1 The Experimental Design: Two Axes, Four Variants

Every experiment in this thesis compares encodings of the same problem that differ in how the domain-specific heuristic is represented and interpreted. The rules that define the problem, its guess and its constraints, are held fixed across the variants of a family; what varies are the two properties that Section 1.4 and Section 1.5 identified as the points on which clingo and Alpha disagree, and they vary independently.

The *approach* axis fixes where the heuristic is evaluated. In the ground-and-solve approach the heuristic is written with native `#heuristic` directives: Gringo expands them before search and Clasp's Domain heuristic consumes the resulting ground objects. In the lazy approach the heuristic remains a handful of ground facts, `__heuristic(...)` for the native backend and `heuristic("...")` for the Prolog one, which the propagator reads at its initialization and evaluates at every decision step against the partial assignment.

The *semantics* axis fixes how a heuristic condition reads partial information: the clingo-like semantics, under which `not a` holds only once a has been assigned false and an aggregate has a value only once its sources are determined, against the Alpha-like semantics, under which `not a` holds as soon as a is not assigned true, and an aggregate is computed over the atoms currently true. Only the second reading makes a heuristic dynamic in the sense of reacting to a partial assignment during search, as motivated in Section 1.1: an instruction such as "prefer the element whose insertion keeps the two sums balanced" presupposes that the sum can be evaluated against an assignment that is still being built; under clingo's semantics, that sum simply has no value until it is too late for the guidance to matter.

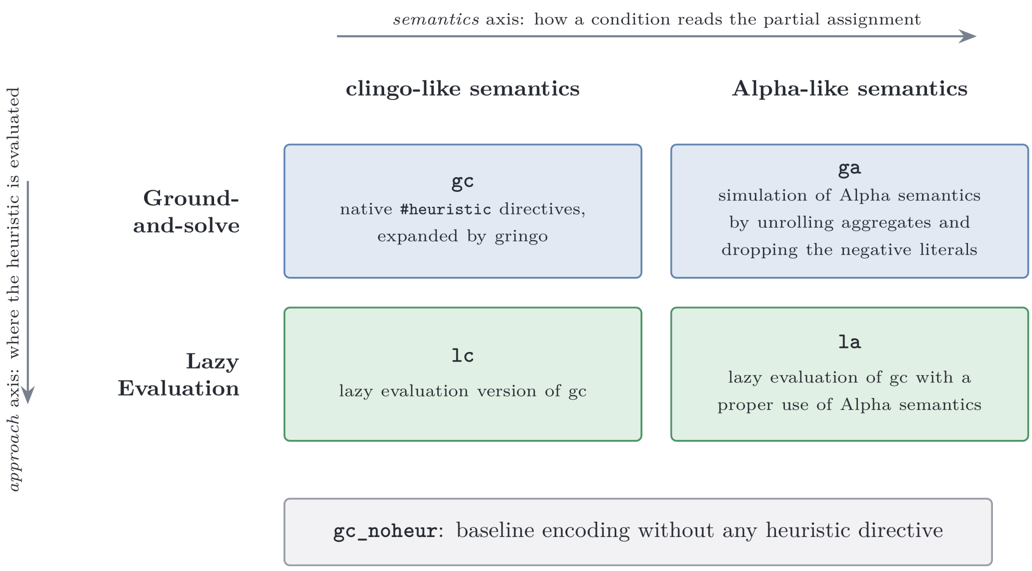


Figure 2.1: The experimental matrix. Prefixes name the approach, suffixes the semantics.

Crossing the two axes yields the four variants that the rest of the thesis refers to by: `gc` and `ga` for the ground approach under the clingo and the Alpha semantics, `lc` and `la` for the lazy evaluated one, the prefix naming the approach and the suffix the semantics. To them is added the heuristic-free baseline `gc_noheur`, which carries the same domain logic with no heuristic at all and measures what the solver does when it is left alone (Figure 2.1).

To isolate specific architectural and modeling mechanisms beyond this primary matrix, the experimental design also incorporates three targeted exploratory variants: `ga_weak`, `la_aux`, and `la_co`. Their empirical motivation and methodological role in disentangling the contributions of default negation, body materialization, and constraint formulation are discussed in Section 2.1.2.

2.1.1 Simulating Alpha in Ground clingo

Simulating Alpha-like semantics within a ground-and-solve pipeline requires addressing two distinct semantic aspects: default negation over unassigned literals and dynamic aggregate evaluation.

The first transformation drops negative literals that test the partial state (such as `not c(X)` in BSP, `not not_assigned_sensor(S)` in PUP, and guards like `not fullCabinet(C)` in HRP). Under Alpha-like semantics, these literals are satisfied by unassigned atoms as well as false ones. Since a target atom is unassigned at the moment a decision is evaluated, dropping the negative literal altogether provides the closest approximation in ground clingo; this is harmless in practice because the solver never re-decides an atom that is already assigned.

The second transformation replaces the equality condition with a *lower/upper* bound. For BSP:

```
sum_value(0..tot).
```

```
#heuristic b(X) : x(X), sum_value(S), S <= #sum{Y : c(Y)},
    W = ((n+1)*S)+X. [W, true]
```

Instead of binding S to the final value of the sum, the directive is unrolled over all candidate values `sum_value(S)` and guarded by the condition $S \leq \#\texttt{sum}\{Y : c(Y)\}$. During search, as soon as the partial sum reaches a bound S, every directive with that bound becomes applicable; among the applicable ground directives for the same atom, Clasp's Domain heuristic retains the one with the maximum weight, which corresponds exactly to the *current* value of the aggregate. The max-weight selection thus reconstructs the dynamic aggregate at solving time. For aggregates whose contribution to the weight decreases rather than increases (such as the free-capacity terms N_0, N_1, N_2 in PUP) the same transformation is applied with the bound $V \geq \#\texttt{max}\{\ldots\}$, so that the max-weight selection picks the smallest proven value.

This full simulation in `ga` is faithful, but its price is exactly the combinatorial explosion that motivated the thesis: one ground directive per point of the product of the unrolled domains, measured in Chapter 4.

2.1.2 Methodological Rationale for the Exploratory Variants

While the four main variants evaluate the core matrix of approach versus semantics, three targeted exploratory variants are introduced as experimental controls to isolate specific factors:

Isolating Alpha-Negation from Dynamic Aggregates (`ga_weak`). Alpha-like semantics treat default negation as satisfied over unassigned literals, and evaluate aggregates dynamically over currently true atoms. While `ga` simulates both via extensive ground unrolling, `ga_weak` acts as an ablation study that applies Alpha's negation semantics while avoiding unrolling aggregates. This investigates whether adapting default negation alone provides any improvement during search, or whether dynamic aggregate evaluation is strictly indispensable for an effective search guidance.

Deconstructing Lazy Efficiency: Directives vs. Body Materialization (`la_aux`). Lazy evaluation replaces thousands of ground `#heuristic` directives with a minimal set of declarative facts. This raises a natural question: does the performance gain come merely from having fewer heuristic directives, or from keeping the calculation of heuristic conditions entirely out of the ground program? The `la_aux` variant tests this distinction by pre-materializing all heuristic conditions into auxiliary ASP rules (`h_b(X,S)`), while querying them through the lazy propagator.

Disentangling Heuristic and Problem Constraint Bottlenecks (`la_co`). In the baseline BSP encoding without heuristics, the balance constraint generates a combinatorial explosion of ground rules. Consequently, even though lazy evaluation reduces search time, total execution remains dominated by grounding the base problem constraints. The `la_co` variant combines the lazy evaluation with a compact linear formulation of the balance constraint.

2.1.3 Weights, Priorities, and Flattening

Section 1.5 recorded that the two systems read the annotation $[W@P]$ of a heuristic differently. In Alpha, the pair is an *absolute* rank over all heuristic instances, read lexicographically as (P, W): the priority P is a global level and decides first, so no instance of level P is considered while an instance of a level $P' > P$ is still applicable, and the weight W only orders the instances that share the same level. In clingo, the evaluation is different: P is local, arbitrating only between heuristics relative to the *same* target atom, while the order in which distinct atoms are decided comes from the weight alone. An encoding written for Alpha therefore cannot be handed to clingo verbatim.

The intended order can nevertheless be recovered, by *flattening* the level into the weight:

$$W_{\mathit{flat}} \;=\; W + P \cdot M, \qquad M > \max W - \min W,$$

where the intra-level weights W range over $[\min W, \max W]$ and the level multiplier M is larger than that range. Under this condition $W + P \cdot M > W' + P' \cdot M$ holds whenever $P > P'$, regardless of the two individual weights, and reduces to $W > W'$ when $P = P'$: the total order induced on the atoms is then the one Alpha's absolute levels induce. When all weights are non-negative, which is the case in every encoding of this suite, $M > \max W$ suffices.

Each benchmark fixes its own M. In the PUP encodings M is the constant offset 100,000 added to the zone weights, against a maximum sensor weight of 26,220 on the benchmark instances; in HRP M is derived from the instance as `levelMul(LM) :- LM = MC + MR + 10` over the maximum cabinet and room identifiers, which gives 16 on the smallest instance and 54 on the largest. BSP doesn't need a multiplier, as discussed in Section 3.3.1.

2.1.4 Problems on bigger weights

Clasp stores the weight of a `#heuristic` modification in a signed 16-bit integer field and clamps any value exceeding 32,767. Consequently, two ground directives whose weights both exceed 32,767 become indistinguishable to Clasp's Domain heuristic. A flattened weight must therefore satisfy $W + P \cdot M \leq 32{,}767$ for the ground variants to rank as intended. HRP stays far below this bound; PUP exceeds it on the zone directives, with the consequences discussed in Section 4.10; and BSP, whose maximum weight grows as $\Theta(n^3)$, crosses it within the benchmarked range. What matters there is not the largest weight the grounder emits, but the largest one the search actually reaches: directives whose aggregate value is never attained may clamp without any observable effect. On BSP that effective crossing occurs at $n = 51$, as analyzed in detail in Section 4.7. The lazy backends are not subject to this bound because they rank targets internally and hand Clasp a ready decision rather than a weight.

The suite adopts clingo's convention in *all four variants and in both backends*: absolute levels always live in the weight, simplifying the propagator as it does not need to account for a separate priority field. Once global levels are folded into weights, weight becomes the sole ordering criterion under both semantics. Accordingly, the native lazy language (Section 2.3) omits the priority field entirely.

A last convention concerns empty aggregates: `#max` over an empty set is 0 in Alpha but $-\infty$ (`#inf`) in clingo. All encodings therefore write `#max{0; ...}` with an explicit neutral element, and both lazy backends implement the same rule (the native `MaxState` returns 0 when empty; the Prolog runtime defines `dyn_max` with a 0 default).

2.2 System Architecture

The project maintains two modified copies of clingo. They share the same upstream sources and differ only in their extensions: the build in `clingo-native` carries the C++ backend, while the build in `clingo-prolog` carries the Prolog backend, configured with `clingo_USE_SWIPL=ON` so that the SWI-Prolog engine is linked in-process, making its memory impact easier to account for. In both builds, the extension lives entirely under `libclingo`, exposes the same class name, and is registered at the same point of `clingo_app.cc`:

```
HeuristicPropagator my_propagator;
ctl.register_propagator(my_propagator, true);
```

The propagator is registered on *every* run of the modified binary. On a program that contains no heuristic facts, it remains inert because `init` finds nothing to watch and `decide` always returns 0; the same binary therefore runs the ground variants and the heuristic-free baseline without requiring a separate build. The second argument, `true`, instructs Clasp to call the propagator sequentially, avoiding race conditions on the incremental state described in Section 3.1.1. No other parts of clingo are modified. Figure 2.2 shows the architecture of the two binary build variants.

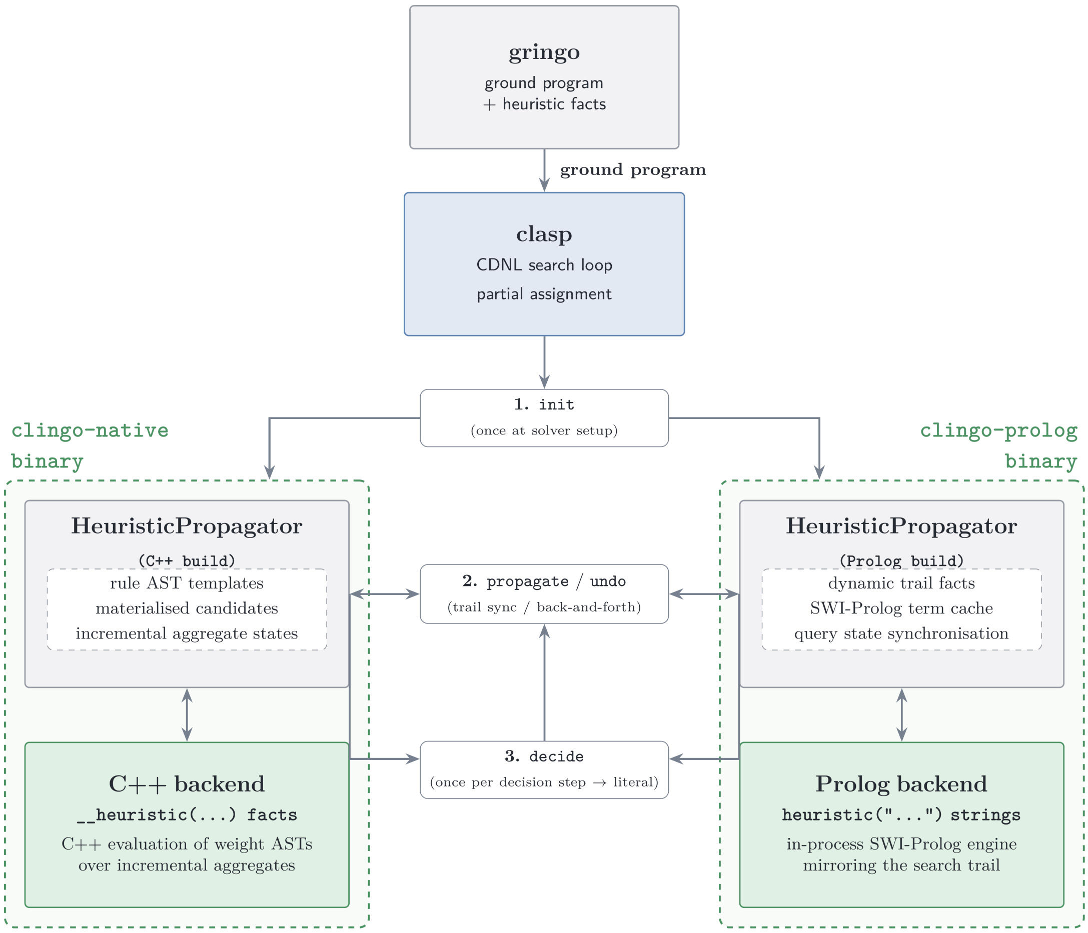


Figure 2.2: System architecture of the two binary variants, showing their respective heuristic propagators and evaluation backends attached to Clasp.

In both builds, the class implements `clingo::Heuristic`—the propagator interface extended with the `decide` callback (Section 1.6). Both backends follow the same four-step lifecycle:

- During `init`, they scan symbolic atoms for trigger facts (`__heuristic(...)` in the C++ build and `heuristic("...")` in the Prolog build), construct their runtime representation, and register watches on solver literals.
- During forward search, `propagate` updates internal aggregate and candidate states as watched literals become true.
- During backtracking, `undo` restores internal aggregate and candidate states as literals are retracted from the trail.
- At each decision point, `decide` either returns a signed literal for the highest-ranked target or returns 0, delegating the decision to Clasp's fallback heuristic.

To activate this mechanism, runs are launched with the flag `-heuristic=Domain`. A detailed discussion of the architectural differences between the two backends is provided in Chapter 3.

The propagator never adds a constraint, never derives a literal, and never refuses one. It only reorders the decisions Clasp would have taken anyway, so the answer sets are untouched, and a heuristic can make a run slower or faster but cannot change the meaning of the encoding (detailed in Section 3.1.3).

2.3 Translation of the Heuristic Directives

Neither backend reads native `#heuristic` directives. Instead, a custom declarative "lazy heuristic" language replaces them with a single ground fact that *describes* the rule. This section presents what one may write, and what each construct means, by deriving that fact for the BSP heuristic of Section 1.1 one component at a time. How the description is executed is left to Section 3.

The starting point is the directive as clingo reads it:

```
#heuristic b(X) : x(X), not c(X), S = #sum{Y : c(Y)}, W = ((n+1)*S)+X. [W, true]
```

Target. The target atom is `b(X)`, where X ranges over the ground instances. The lazy fact specifies only the predicate name, `__target(b)`. During `init`, the propagator queries clingo's atom table for all ground instances of `b(...)` already materialized by Gringo.

Positive body. The condition `x(X)` shares its argument with the target: the candidate for `b(7)` is alive only while `x(7)` is true. A condition of this shape is written as the bare predicate name, `x`, and the propagator matches its arguments against the target tuple position by position.

However, a body predicate may have a different arity; its arguments may sit in different positions; or a value may have to be read out of it and used in the weight. The correspondence is then stated explicitly:

```
__body(pred, __match(src, tgt), __bind_arg(var, idx))
```

`__match(src, tgt)` requires the argument at the `src` position of the body atom to equal the argument at the `tgt` position of the target atom, where both `src` and `tgt` are integers. All indices are 0-based, and at least one `__match` is mandatory. `__bind_arg(var, idx)` extracts argument `idx` of the body atom into a variable.

Given the ground facts:

```
score(1,10). score(2,5). score(3,7).
```

and the target atoms:

```
choose(1). choose(2). choose(3).
```

the directive:

```
__heuristic(__target(choose),
            __body(score, __match(0,0), __bind_arg(w,1)),
            __weight(w), __modifier(true)).
```

pairs each `choose(X)` with the `score(X,W)` that agrees with it on the first argument, and binds `w` to the second argument of that atom. The weights are then 10, 5 and 7, so the propagator decides `choose(1)` first.

Negative body. The condition `not c(X)` likewise matches its arguments against the target tuple. Negative body literals are specified by prefixing the predicate name with `__n_`.

Semantics. The reading of that negation is fixed by the `__semantics(S)` selector, which decides whether `__n_c` means "`c(X)` is assigned false" or "`c(X)` is not assigned true". The same selector fixes the reading of aggregates, described below. It takes `alpha` or `clingo`, and defaults to `clingo`, so that a directive written without it behaves as the native `#heuristic` it replaces.

Aggregates. Aggregates are evaluated lazily during search. `__bind(s, __sum(c))` binds variable `s` to the sum over the predicate `c` (by default aggregating its first argument). The operators `__count`, `__min`, and `__max` follow the same structure.

An aggregate can be restricted *per target* using `__filter(src_idx, tgt_idx, offset)`, which keeps only source atoms whose argument at position `src_idx` equals argument `tgt_idx` of the target plus an optional `offset`. For instance, in PUP, retrieving the maximum number of sensors on the *previous* unit ($U-1$) for a target `assigned_zone_unit(Z,U)` is expressed as:

```
__bind(m1, __max(num_sensors_on_unit, 0, __filter(1, 1, -1)))
```

where `0` selects the aggregated argument N in `num_sensors_on_unit(N, U')`, and the filter matches $U' = U - 1$.

Weight. Weight is expressed as a *term tree* such as:

```
__weight(__add(__mul(s, B), self))
```

which represents $(s \cdot B + \mathit{self})$, where `self` is a shorthand for the target atom's argument at index 0 without requiring a variable binding.

The binary constructors `__add`, `__sub`, and `__mul` form expression trees whose leaves belong to three categories: integer constants, variables of the heuristic language, and target arguments (accessed through `self` or `__bind_target_arg(var, idx)` for any other).

At `init`, the propagator compiles all weight term trees into ASTs. During search, each active heuristic object evaluates its weight AST dynamically against the current trail state, avoiding the memory overhead of materializing them in the ground program. Only candidates with a strictly positive resolved weight are taken into account; the others are ignored, leaving their decision to the solver's default heuristic.

Modifier. The `__modifier(M)` selector specifies which of clingo's heuristic modifiers to use: `level` sets the ranking value alone, `sign` sets the preferred polarity alone, while `true` (the default) and `false` set both.

The solver's `decide` interface expects a *signed* literal, so a ranking value alone does not determine a decision. Each target keeps one slot per channel, ranking value and preferred polarity, and when several directives write to the same slot the one with the highest weight wins. This is what replaces clingo's local priority. Across the benchmarks all directives use `true`, except for the final layer of HRP, which uses `false` to prune the remaining choice atoms.

Assembling the components gives the fact in full:

```
__heuristic(__target(b), x, __n_c, __bind(s, __sum(c)),
            __weight(__add(__mul(s, B), self)),
            __semantics(alpha), __modifier(true)) :- B = n + 1.
```

construct	**clingo directive**	**native lazy fact**	**Prolog rule string**
structure			
target	`b(X)`	`__target(b)`	`heuristic(b(X), W, P,` `M)`
positive body, target tuple	`x(X)`	`x`	`holds(x(X))`

positive body, join	`p(X,V)`	`__body(p,`
		`__match(0,0),`
		`__bind_arg(v,1))`
			`holds(p(X,V))`

negation	`not c(X)`	`__n_c`	`alpha_not(c(X))` `clingo_not(c(X))`

aggregate	`S = #sum{Y :`
	`c(Y)}`
		`__bind(s,`
		`__sum(c))`
		also `__count`, `__min`,
		`__max`
			`aggregate_all(`
			`sum(Y), holds(c(Y)),`
			`S)`

filtered aggregate	`M = #max{N :`
	`q(N,U-1)}`
		`__bind(m,`
		`__max(q, 0,`
		`__filter(1,1,-1)))`
			`U1 is U-1,`
			`dyn_max(`
			`holds(q(N,U1)),`
			`N, M)`

target arguments	`X`, `Y` in `b(X,Y)`	`self`,
		`__bind_target_arg(`
		`y, 1)`
			ordinary head variables

annotations			
weight	`W = ((n+1)*S)+X`	`__weight(__add(` `__mul(s,B), self))`	`W is B*S+X`
modifier	`[W, true]`	`__modifier(true)`, default `true`	fourth head argument
semantics	— (fixed by the system)	`__semantics(alpha)` `__semantics(clingo)`	choice of `alpha_not` / `clingo_not`
target still free	— (implicit in clasp)	implicit: an assigned target is skipped	`target_available(` `b(X))`

Table 2.1: The same heuristic in the three notations.

Chapter 3

Implementation and Benchmark Setup

3.1 Inside the Native Backend

3.1.1 Parser and Runtime Representation

The parser in `libclingo/src/heuristic_parser.cc` turns each `__heuristic` symbol into a `HeuristicRuleTemplate`: target atom, positive and negative body literals, variable bindings, modifier, semantics, and an arithmetic AST for the weight.

During `init`, the propagator inspects the ground symbolic atoms and builds an index mapping each predicate and its argument tuple to the corresponding solver literal. It then materializes candidates: for each ground target atom and matching combination of positive body atoms, it creates a `RuntimeHeuristicCandidate` storing the target literal, target tuple, body literals, bound variable values, and instantiated aggregate keys.

In BSP with $n = 100$, this yields only 100 candidates for `b(X)` (and 200 across both partitions), whereas the standard ground encoding generates 1,010,200 directives ($2 \times 100 \times 5051$) to cover all possible discrete sums $S \in \{0, \ldots, 5050\}$. Thus, candidate materialization in the lazy propagator scales linearly with the number of ground targets rather than with the domain size of dynamic aggregates, eliminating exhaustive grounding.

Finally, the propagator registers watches exclusively for literals that affect heuristic state: both polarities of target and negative body literals, positive body literals, and aggregate source literals.

3.1.2 Incremental Aggregates

Aggregate states implement three methods: `add`, `remove`, and `result`.

Sums and counts (`SumState`, `CountState`) store a single `int`, because adding and removing values can be done with simple addition and subtraction. In contrast, min and max (`MinState`, `MaxState`) cannot be undone with arithmetic; when the current minimum or maximum is removed during backtracking, the state must know what the next best value was. They therefore use an `std::map<int, int>` that maps each active value to how many times it occurs.

Each source atom updates the `AggregateState` matched by its `RuntimeAggregateKey`: values are added in `propagate` and removed in `undo`.

Under *clingo* semantics, an aggregate is evaluated only after all its source literals are assigned, gating dependent candidates until then. Under *Alpha* semantics, candidates directly read whatever partial aggregate value is currently available on the trail.

3.1.3 Evaluation and Decision

A candidate is *active* when its target literal remains unassigned, all its positive body literals evaluate to true, and every negative body literal is satisfied under the template's semantics. The two supported semantics differ only in how they treat negative literals:

```
static bool negative_literal_is_satisfied(HeuristicSemantics s,
    clingo::TruthValue v) {
    return s == HeuristicSemantics::clingo ? v == clingo::TruthValue::False
                                           : v != clingo::TruthValue::True;
}
```

Once a candidate satisfies its body conditions, the propagator computes its numerical weight by evaluating its weight AST over the current aggregate values and term arguments. The resulting effect is then applied to the target literal's `TargetHeuristicState`.

The native propagator adopts four of clingo's domain heuristic modifiers:

- `level`: sets the branching priority to w (higher values are chosen first), leaving the truth polarity unchanged;
- `sign`: sets the preferred truth polarity based on the arithmetic sign of w (positive for *true*, negative for *false*), using $|w|$ to resolve precedence among conflicting sign directives;
- `true`: sets the branching priority to w and forces the truth polarity to *true*;
- `false`: sets the branching priority to w and forces the truth polarity to *false*.

clingo's remaining modifiers, `init` and `factor`, are not supported because custom propagators cannot manipulate Clasp's internal VSIDS score tables.

If multiple heuristic rules target the same atom, their effects are combined into a `TargetHeuristicState` that tracks the `level` and `sign` across all active `RuntimeHeuristicCandidate` instances.

Unassigned targets with an active `level` are stored in an ordered set:

```
std::set<DecisionRankKey, DecisionRankGreater> active_decision_ranks_;
```

Entries are sorted by decreasing weight. Because weights are stored as standard 32-bit integers, this avoids the 16-bit saturation limit of Clasp's internal domain table (Section 2.1.3).

When Clasp calls `decide`, the propagator inspects `active_decision_ranks_` from the beginning, discarding any targets that are already assigned on the trail or whose weights have changed. It then branches on the first valid target using its stored `sign`. If the set is empty, the propagator checks whether Clasp's fallback atom has a stored `sign`; otherwise, it returns 0 to delegate the decision to Clasp. During backtracking, `undo` restores aggregates and candidate states using `noexcept` functions to prevent state corruption during solver unwinding.

3.2 The Prolog Backend

While the native backend prioritizes evaluation speed through compiled C++ structures, the Prolog backend evaluates heuristics as arbitrary logic programs via an embedded engine, trading runtime overhead for unrestricted expressiveness. This section describes its runtime architecture and trail synchronization mechanisms (with a comprehensive performance comparison presented in Section 4.13).

3.2.1 Rule Strings and Normalization

In the Prolog backend, a heuristic is defined as a rule string whose head is a four-argument term: `heuristic(Target, Weight, Priority, Modifier)`. The BSP heuristic from Section 2.3 is specified as follows:

```
heuristic("heuristic(b(X), W, 0, true) :-
    aggregate_all(sum(Y), holds(c(Y)), S), holds(heuristic_base(Base)),
    holds(x(X)), target_available(b(X)), alpha_not(c(X)),
    W is Base*S+X.").
```

The rule body uses standard Prolog syntax and maps directly to the native constructs: `holds/1` for positive conditions, `alpha_not/1` for negative conditions, `aggregate_all/3` for dynamic sums, and `is/2` for arithmetic evaluation of weights. Furthermore, `target_available/1` ensures the target atom is still unassigned. While the native backend checks this implicitly during candidate selection, the Prolog rule must state it explicitly to prevent proposing atoms that are already assigned on the trail.

During initialization, the propagator collects all `heuristic/1` facts, normalizes the rule strings, and classifies their semantics: rules containing `clingo_not(...)` use clingo semantics, while all others use Alpha semantics. The propagator extracts the predicate signatures referenced in the rules (`holds`, `alpha_not`, `clingo_not`, `target_available`, and rule targets) and asserts into Prolog only solver atoms matching those signatures, avoiding unnecessary memory and synchronization overhead from irrelevant predicates.

3.2.2 The Runtime Program

During initialization, the backend starts an embedded SWI-Prolog engine [13] and loads a set of base Prolog rules:

```
holds(A) :- true_atom(A).
holds(A) :- static_atom(A), \+ true_atom(A).
clingo_not(A) :- false_atom(A).
alpha_not(A)  :- \+ holds(A).
target_available(A) :- target_atom(A), \+ true_atom(A), \+ false_atom(A).
dyn_max(Goal, Template, Max) :-
    (aggregate_all(max(Template), Goal, R) -> Max = R ; Max = 0).
```

As Clasp makes decisions, the propagator mirrors the trail in the Prolog database by adding `true_atom(A)` when an atom is assigned to true and `false_atom(A)` when it is assigned to false, removing them upon backtracking. Undecided atoms simply do not exist in the database. Because of this, `alpha_not(A)` succeeds whenever an atom does not hold true. In contrast, `clingo_not(A)` requires an explicit `false_atom(A)` on the trail. In the same way, `aggregate_all` dynamically computes sums and counts over the atoms currently true on the trail. `target_available(A)` verifies that a target is neither true nor false before branching on it, and `dyn_max/3` defaults to 0 when evaluating an empty set (Section 2.1.3).

3.2.3 Synchronization and Decision

Trail synchronization is incremental: during search, `propagate` and `undo` update the state of atoms in the Prolog database as their solver literals change on the trail.

At each decision point (`decide`), the backend queries all solutions to `heuristic(T, W, P, M)`, filters out targets that are already assigned on the trail, and branches on the candidate with the highest priority and weight. Unlike the native C++ backend, which incrementally maintains a sorted set of active targets, the Prolog backend re-evaluates the heuristic query from scratch at every decision step. This design prioritizes relational expressiveness over per-decision efficiency (evaluated in Section 4.6).

3.3 Benchmark Problems and Encodings

3.3.1 Balanced Sum Problem (BSP)

The BSP divides the set of integers $\{1, \ldots, n\}$ into two disjoint sets, `b/1` and `c/1`, such that their sums differ by at most one. The heuristic greedily assigns unassigned integers to set b with a priority proportional to the current sum of set c, driving the solver toward balanced partitions. Because all decisions share a single priority level, the current sum S acts as the primary ranking term, with the element value X resolving ties between equal sums.

The implementation of the three exploratory variants introduced in Section 2.1.2 on BSP required some specific adjustments:

- `ga_weak`: The ground heuristic directives omit the negative body guards (`not c(X)` / `not b(X)`), retaining only the static aggregate binding `S = #sum{Y : c(Y)}` without unrolling over candidate values:

```
#heuristic b(X) : x(X), S = #sum{Y : c(Y)}, W = ((n+1)*S)+X. [W, true]
#heuristic c(X) : x(X), S = #sum{Y : b(Y)}, W = ((n+1)*S)+X. [W, true]
```

Without the `sum_value(S)` domain predicate, Gringo generates exactly $2n$ ground directives in total.

- `la_aux`: Heuristic conditions and aggregate values are pre-materialized:

```
h_b(X,S) :- x(X), not c(X), S = #sum{Y : c(Y)}.
h_c(X,S) :- x(X), not b(X), S = #sum{Y : b(Y)}.
```

The lazy propagator language is then instructed to monitor these auxiliary atoms via the `__body` constructor:

```
__heuristic(__target(b), __body(h_b, __match(0,0), __bind_arg(s,1)),
            __weight(__add(__mul(s, B), self)),
            __semantics(alpha), __modifier(true)) :- B = n + 1.
```

The propagator will track truth-value assignments over the ground `h_b(X,S)` atoms on the trail, testing whether auxiliary trail synchronization reintroduces solver overhead.

- `la_co`: uses the same lazy heuristic as `la`, but instead of computing and comparing the two sums separately:

```
sumB(Sum) :- Sum = #sum{Y : b(Y)}.
sumC(Sum) :- Sum = #sum{Y : c(Y)}.
:- sumB(SB), sumC(SC), (SB - SC)**2 > 1.
```

it enforces the partition balance directly with a single aggregate rule:

```
#const min_val = tot / 2.
#const max_val = (tot + 1) / 2.
:- not min_val <= #sum{ Y : b(Y) } <= max_val.
```

This removes the heavy grounding bottleneck of the base problem while keeping the meaning of the heuristic unchanged.

3.3.2 Partner Units Problem (PUP)

The PUP [14] assigns communication units to interconnected zones and sensors under capacity and topology constraints. The heuristic uses a two-level priority policy: zones are assigned first, followed by sensors.

The base encoding is taken from the Alpha reference literature [14, 9]. However, because the original encoding was written for a lazy solver, running it directly in clingo causes a severe grounding bottleneck. To mitigate this while preserving exact solution equivalence, the encoding was refactored using standard ASP modeling practices: bounding unit indices to valid ranges, expressing capacity limits via linear `#count` aggregates, and eliminating a redundant blocking rule that generated excessive ground constraints.

Finally, the global priority of zones over sensors is flattened into the heuristic weights using the +100,000 offset introduced in Section 2.1.3.

3.3.3 House Reconfiguration Problem (HRP)

The HRP [15] places items into cabinets and assigns cabinets to rooms under person-ownership and dimensional constraints, minimizing deviations from an existing legacy setup. The heuristic enforces a five-level priority policy: reusing legacy components, placing long items, placing short items, assigning cabinets to rooms, and finally setting unassigned choice atoms to false.

Unlike BSP and PUP, HRP contains no aggregates, isolating the behavior of lazy default negation (`la` vs. `lc`) on partial assignments.

3.3.4 Alpha as an External Reference

We benchmark our propagator against the lazy-grounding solver Alpha [7, 8] as an external state-of-the-art reference. Specifically, we test the *Qh* (Query Heuristics) variant developed by Comploi-Taupe et al. [9], invoked with `-uqh`, which evaluates domain heuristics and dynamic aggregates over partial assignments via Prolog queries. This mechanism directly mirrors our Prolog backend and implements the semantics our native propagator reproduces.

The choice of build matters: the official Alpha 0.7.0 release does not support `#heuristic` directives, and the main upstream branch evaluates aggregates statically. We therefore use the authors' research Qh build, which implements dynamic aggregates over partial assignments. All Alpha encodings and benchmark instances are taken directly from the authors' published materials.

We evaluate Alpha under two configurations on all benchmark families: `alpha_qh` (with domain heuristics enabled via `-uqh`) and `alpha_noheur` (the baseline solver without heuristics, `-ids`). On HRP, we also run `alpha_dom`, which evaluates domain heuristics natively without the Prolog bridge. In Section 4.12, we compare these runs against our native clingo variants—`la` (lazy propagator), `ga` (ground unrolled simulation), and `gc_noheur` (heuristic-free baseline)—to isolate the cost of lazy grounding from the benefit of dynamic heuristic guidance.

Search counters cannot be compared across systems: Alpha's *guesses* and *backtracks* measure lazy CDNL search interleaved with dynamic rule instantiation, whereas clasp records decision *choices* and *conflicts* over a pre-ground program. Cross-system comparisons are therefore limited to external process metrics recorded by `runlim`: wall-clock time and peak resident set size (RSS). Furthermore, because Alpha runs on the Java Virtual Machine (JVM), its peak RSS includes the heap pre-allocated via `-Xmx`, providing an upper bound on operational memory rather than the state held by the algorithm itself.

3.4 Benchmark Infrastructure and Metrics

The experimental campaigns are orchestrated using `benchmark-tool` [16], the Potassco benchmarking suite designed to automate ASP experiment execution and distribute jobs across SLURM clusters. Each run executes a specific (system, setting, instance) configuration under the supervision of `runlim` [17], which samples resource consumption while enforcing hard wall-clock time and memory limits.

All clingo runs are executed with domain heuristics enabled (`-heuristic=Domain`) and search configured to find a single answer set (`-n 1`). For each run, we record total execution time, solving and grounding components, peak resident memory (RSS), and search statistics including choices and conflicts. We also track the size of the ground heuristic component, comparing materialized directives against lazy facts, along with propagator-specific overheads such as decision calls, query evaluation times, and trail synchronization costs.

Memory is measured as the peak resident set size (RSS) of the entire solver process, which for the Prolog backend includes the embedded SWI-Prolog runtime. Because the engine incurs a constant initialization overhead, lazy Prolog runs can consume more memory than ground variants on small instances; however, it still provides a substantial asymptotic memory advantage on larger instances compared to the ground-and-solve variants.

Ground variants count directives materialized by Gringo, whereas lazy variants count template facts that expand into candidates dynamically during search. The runtime effort deferred by lazy grounding is thus captured directly through the per-decision propagator metrics.

Chapter 4

Results

4.1 Experimental Setup and Dataset

All experiments were executed on dedicated HPC cluster nodes requesting 4 cores, with hard limits of 600 seconds and 30 GB of RAM per run enforced by `runlim`. Both backends were compiled on the test hardware and the version of SWI-Prolog used was 10.0.2.

The evaluation covers the benchmark suite defined in Section 3.3:

- BSP: $n \in \{10, 20, \ldots, 200\}$ (20 instances);
- PUP: `double-`N, with $N \in \{20, 40, \ldots, 200\}$ (10 instances);
- HRP: `house-`N, with $N \in \{2, 4, \ldots, 20\}$ persons (10 instances).

Across all benchmark families, timeouts represent the exclusive first failure mode (denoted $T@N$ in the summary tables), as every failing variant encounters the 600-second time limit before exhausting the memory ceiling. Memory exhaustion is confined to very large BSP instances ($n \geq 180$) where base program grounding alone exceeds the 30 GB budget, beyond all the first failures for BSP.

On BSP, search trajectories between C++ and Prolog coincide identically across all solved instances. On PUP and HRP there are minor discrepancies in choices that arise solely from the tie-breaking among equal-rank candidates.

4.1.1 Measurement Reliability and Timing Variation

Tables 4.1–4.3 summarize the outcome of the benchmark campaign, reporting the number of solved instances and the first failure point for each variant. In the BSP benchmark, solver failures scale predictably with instance size: once a variant fails at a given size, it also fails for all subsequent sizes, with timeouts representing the first failure mode in all cases (between $n = 70$ and $n = 170$). On PUP, however, difficulty does not scale monotonically with size: `lc` times out on `double-80` yet successfully solves `double-100`. This behavior stems from the benchmark design, which provides only a single instance per size with significant variations in hardness between neighboring sizes; Section 4.10 analyzes this effect in detail using choice counts rather than runtimes.

Variant	native		Prolog	
	solved	first failure	solved	first failure
gc_noheur	15/20	T@160	15/20	T@160
gc	13/20	T@140	13/20	T@140
ga	6/20	T@70	6/20	T@70
ga_weak	14/20	T@150	14/20	T@150
la	**16/20**	**T@170**	**15/20**	**T@160**
lc	15/20	T@160	14/20	T@150
la_aux	6/20	T@70	3/20	T@40
la_co	**20/20**	—	**20/20**	—

Table 4.1: BSP campaign overview (n = 10..200, 600 s / 30 GB per run). Best results in bold; bottom block shows exploratory and reformulated variants.

Variant	native		Prolog	
	solved	first failure	solved	first failure
gc_noheur	4/10	T@100	4/10	T@100
gc	3/10	T@80	3/10	T@80
ga	**10/10**	—	**10/10**	—
la	**10/10**	—	**10/10**	—
lc	4/10	T@80 (non-mon.)	2/10	T@60

Table 4.2: PUP campaign overview (`double-`N, N = 20..200, 600 s / 30 GB per run). Best results in bold.

Variant	native		Prolog	
	solved	first failure	solved	first failure
gc_noheur	10/10	—	10/10	—
gc	10/10	—	10/10	—
ga	10/10	—	10/10	—
la	10/10	—	10/10	—
lc	8/10	T@18	7/10	T@16

Table 4.3: HRP campaign overview (`house-`N, N = 2..20 persons, 600 s / 30 GB per run).

Evaluating these summary tables reveals that the maximum instance size solved provides different insights depending on the problem of choice.

- **On BSP**, the solvability frontier alone is insufficient to differentiate the approaches. Variants sharing the same base encoding (`gc_noheur`, `la`, and `lc`) all stop between $n = 150$ and $n = 160$, constrained by the heavy grounding of the base program. What distinguishes these variants is their internal search efficiency: `la` virtually eliminates solving time and conflicts (Sections 4.3 and 4.6).

- **On PUP**, the "alpha semantic"-like dynamic guidance enables both `ga` and `la` to solve the entire range up to $N = 200$, whereas the other variants stall around $N = 80$ due to search space explosion. With search effectively solved, PUP isolates the cost of the heuristic representation itself: the lazy approach `la` cuts grounding time and peak memory in half compared to the static aggregate unrolling in `ga` (**12.7 GB** vs **24.6 GB** at $N = 200$).
- **On HRP**, the solvability frontier is uninformative because nearly all variants solve the entire benchmark suite; the evaluation centers instead on decision quality and the severe misguidance observed in `lc`.

Some other trends emerge from the benchmarks. First, the static ground simulation (`ga`) fails prematurely on BSP at $n = 70$, nine sizes earlier than the baseline, due to the combinatorial explosion of static unrolling. Second, the native C++ and Prolog backends yield identical frontiers whenever propagator queries remain cheap; where they diverge, the Prolog backend consistently fails earlier due to evaluation overhead, particularly for `lc` where the propagator is queried tens of thousands of times. Finally, `lc` illustrates two distinct failure modes caused by clingo-style aggregate evaluation over the partial assignments: on BSP, the heuristic is completely inactive because its aggregate conditions are never satisfied during search (producing zero candidates), whereas on HRP, delayed evaluation yields misleading suggestions that severely degrade search performance (both detailed in Section 4.13).

4.2 Reduction of the Ground Heuristic Component

The primary goal of this lazy evaluation was to reduce memory spent on materializing heuristics during grounding. This reduction can be directly observed in the number of ground heuristic objects generated as a function of instance size n. In standard ASP encodings, directives such as:

```
#heuristic b(X) : x(X), not c(X), S = #sum{Y : c(Y)}, W = (n+1)*S+X. [W, true]
```

incorporate the dynamic aggregate value directly into the heuristic weight. Because the atoms `c(Y)` are choice atoms whose truth values are determined only during search, the grounder cannot compute the sum ahead of time. It must therefore instantiate a separate directive for every possible (X, S) pair. For $X \in \{1, \dots, n\}$ and S ranging over all reachable subset sums (i.e., integers from 0 to $n(n+1)/2$), the total number of ground directives generated by the two symmetric rules for `b` and `c` is:

$$2 \cdot n \cdot \left(\tfrac{n(n+1)}{2} + 1\right) \;=\; \Theta(n^3).$$

In contrast, the lazy variants completely eliminate this combinatorial expansion at grounding time. The aggregate is evaluated dynamically on the current partial assignment during search, allowing the materialized heuristic representation to remain constant at just two static template facts, regardless of n.

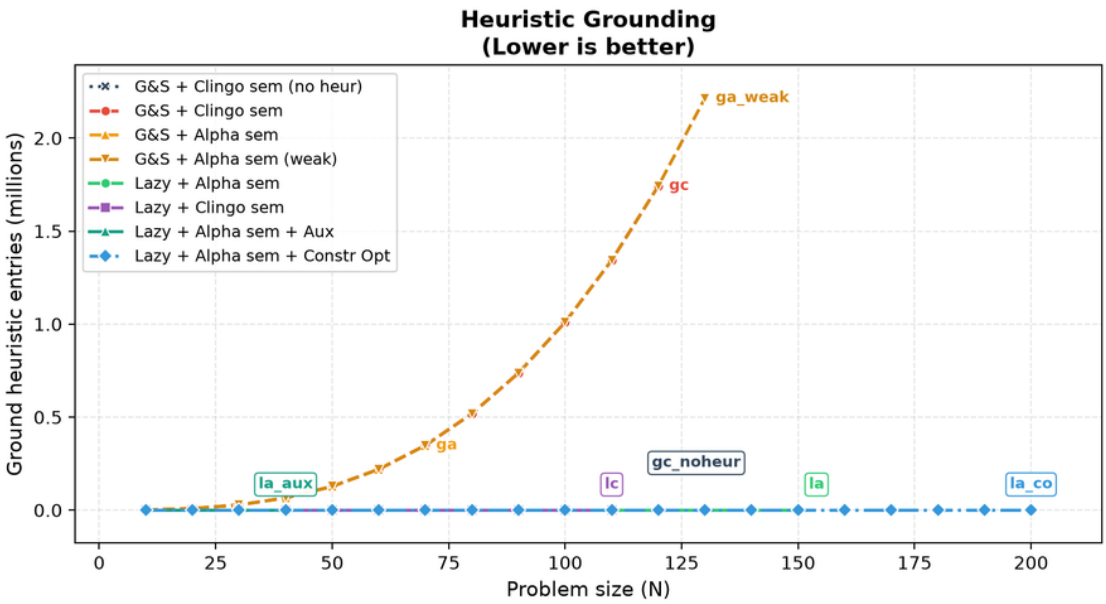


Figure 4.1: Comparison between native ground heuristic objects and lazy heuristic facts.

n	Native heuristic objects	Lazy facts	Ratio
10	1,120	2	560×
50	127,600	2	63,800×
100	1,010,200	2	505,100×
110	1,343,320	2	671,660×

Table 4.4: Growth of the G&S heuristic component against the lazy evaluated one.

This reduction reflects a smaller grounding footprint rather than eliminated computation: lazy evaluation shifts candidate generation to the solver's decision loop, whose runtime cost is analyzed in Section 4.6.

4.3 BSP: Runtime Behaviour

For all variants sharing the base encoding without all the grounded heuristic directives (`gc_noheur`, `la`, and `lc`), the base program grounding alone dominates overall runtime. At $n = 120$, it generates 52.8 million rules, requiring 160.9 s of grounding for `gc_noheur`, 159.7 s for `lc`, and 154.2 s for `la`.

However here we can see a clear difference on the different semantics over the negation: while `gc_noheur` and `lc` require 44.1 s and 47.5 s respectively, `la` completes search in just 0.04 s. As a result, the total runtime curve for `la` remains consistently below all other variants across the entire benchmark, directly reflecting the dramatic solving speedup shown in Figure 4.3.

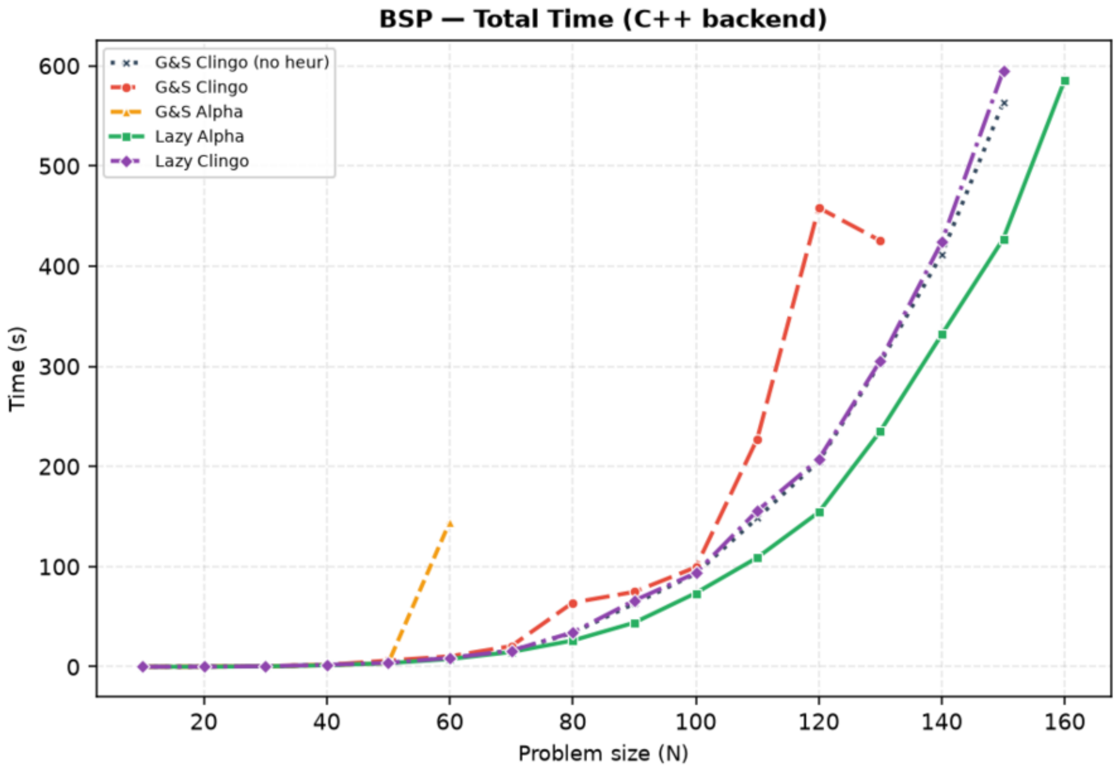


Figure 4.2: One run per point; a curve ends where its variant stops solving within the limits.

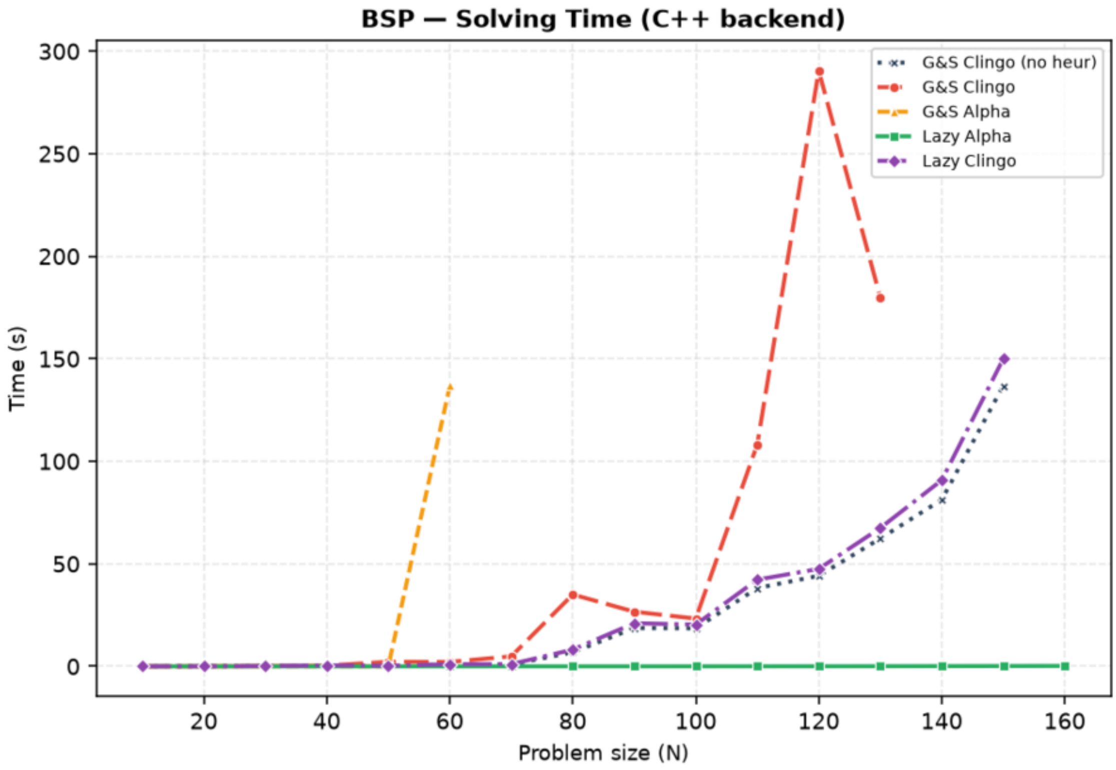


Figure 4.3: `la` solving time is close to zero across all sizes, whereas `ga` degrades sharply at $n = 60$ and times out from $n = 70$ due to clasp's 16-bit weight saturation ($> 32{,}767$).

Variant	Total (s)	Grounding (s)	Solving (s)	Choices	Conflicts
`gc_noheur`	205.1	160.9	44.1	78,565	49,993
`gc`	458.2	167.7	290.5	231,260	160,115
`ga`	T	172.6	> 427.3	694,660	510,480
`ga_weak`	242.1	172.6	69.6	121,159	80,847
`la`	**154.2**	**154.2**	**0.04**	**120**	**0**
`lc`	207.2	159.7	47.5	78,565	49,993
`la_co`	0.009	< 0.01	0.01	117	0

Table 4.5: Execution time and solver statistics on BSP at $n = 120$. For `ga`, solving time represents the last value recorded before timing out.

This highlights three major findings:

1. In `la`, default negation and dynamic aggregate evaluation allow clingo to make use of the heuristic from the very first decision, taking exactly 120 choices with zero conflicts at $n = 120$.
2. While `ga` matches `la` perfectly up to $n = 50$, its performance degrades sharply at $n = 60$ and times out from $n = 70$ onward. This failure is caused by the 16-bit integer weight limit in clasp (32,767), as explained in Section 2.1.4. When weights saturate beyond $n = 50$, dynamic aggregate ranking breaks down entirely.

3. `ga_weak` tracks close to the baseline and is discussed in Section 4.7, while `la_co` is analyzed in Section 4.9.

4.4 BSP: the Two Semantics Under the Microscope

As shown in Figure 4.4 below, `la` requires exactly n decisions with zero conflicts across every benchmarked size n. Under Alpha-like semantics, dynamic aggregate evaluation provides the propagator with usable guidance from the very first decision on an empty assignment, guiding the CDCL solver with perfect choices.

In contrast, `lc` exhibits search statistics that are identical point-by-point to the heuristic-free baseline (`gc_noheur`), recording 78,565 choices and 49,993 conflicts at $n = 120$.

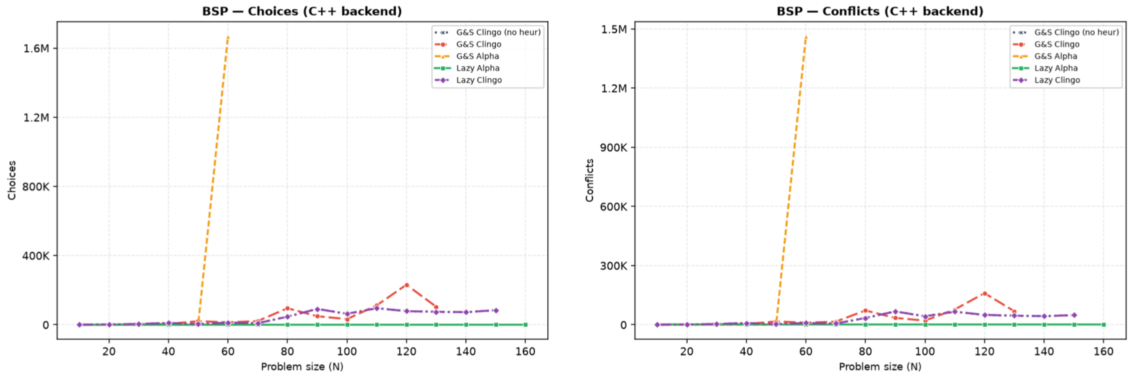


Figure 4.4: BSP search statistics on the C++ backend: solver choices (left) and conflicts (right) across sizes. `la` stays flat at n choices with zero conflicts, while `lc` overlaps the heuristic-free baseline `gc_noheur` point by point.

Under clingo semantics, aggregate guards on `c/1` require all elements to be assigned before the sum can be evaluated and an atom is considered "assigned false" only when clasp demonstrates that it cannot be true. During search, these conditions are virtually never met at decision points. At $n = 120$, although the propagator is consulted on every decision, it yields zero applicable candidates across the entire run. Because both variants run on the exact same native C++ propagator backend, this comparison proves that lazy evaluation alone is insufficient if the underlying semantics cannot evaluate partial information, causing the solver to fall back entirely on default CDCL branching.

4.5 BSP: Rules, Variables, and Memory

The reduction of the heuristic footprint is also evident in the size of the propositional formula generated for the solver. As shown in Table 4.6, at $n = 100$ the two lazy variants replace the 1,010,200 ground directives of `gc` with the two static facts of the DSL, and their variable count (20,300) coincides exactly with the heuristic-free baseline: the heuristic leaves no trace in the propositional formula. The ground variants pay this cost in different forms. In `gc` the directives dominate the formula itself, which grows to more than one million variables, fifty times the baseline; in ga the variable count stays close to the baseline, but the same million directives still have to be grounded, and beyond $n = 50$ this effort does not translate into usable guidance. Only `la_co` falls below the baseline, with 101 variables, because it also reformulates the balance constraint (Section 4.9).

Variant	**Ground Directives / Facts**	**Solver Variables**
`gc`	1,010,200	1,030,606
`ga`	1,010,200	20,402
`ga_weak`	200	20,406
`lc`	2	20,300
`la`	2	20,300
`gc_noheur`	0	20,300
`la_co`	2	101

Table 4.6: Heuristic grounding footprint and propositional solver variables at $n = 100$, C++ backend.

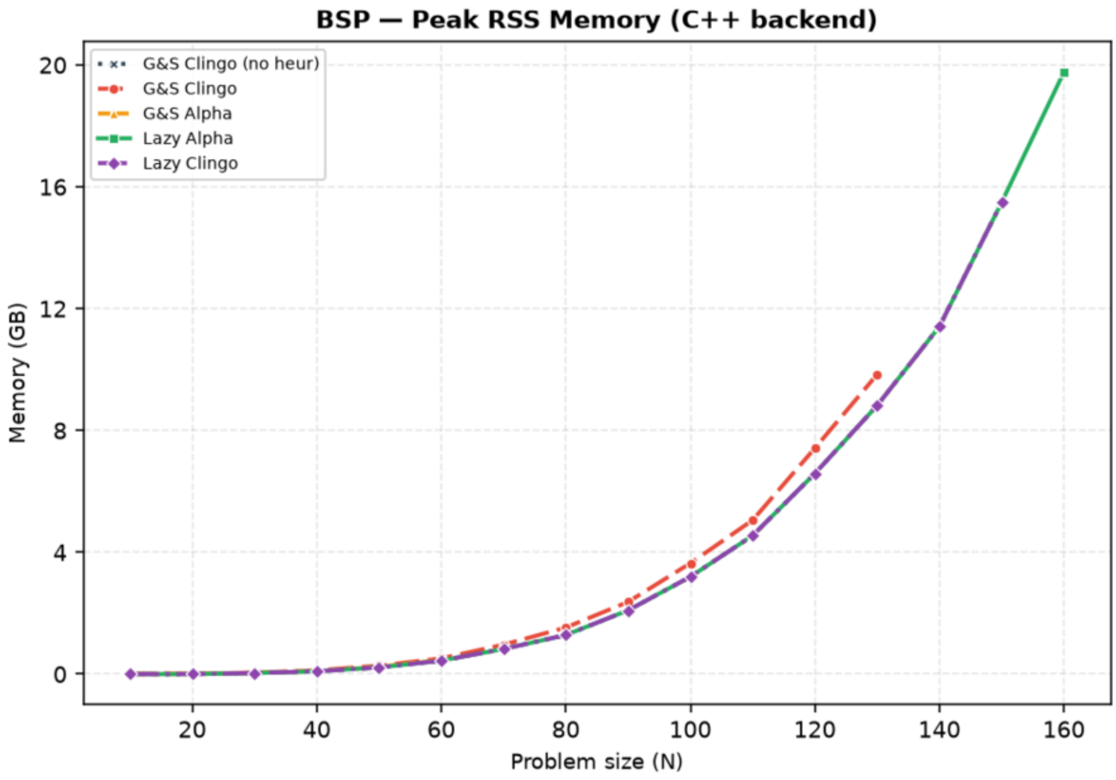


Figure 4.5: Peak Resident Set Size

Figure 4.5 shows that on large instances, memory usage is dominated by the base problem encoding; thus, while lazy heuristics eliminate the memory overhead of heuristic directives, scaling on BSP remains bounded by the ground footprint of the base program, a limitation addressed by constraint reformulation in Section 4.9.

4.6 The Price of Laziness: Per-Decision Cost

The lazy evaluation model represents a space-time trade-off where instead of suffering a combinatorial cost in memory during the grounding phase, there is a cost in computation incurred for each decision made during the search phase. Table 4.7 profiles this cost on both backends.

		Search Activity		**Cost (ms/decide)**		**Prolog Overhead**	
Family	**Var.**	**calls**	**cand./call**	**native**	**Prolog**	**sync (s)**	**prop. share**
BSP n=120	`la`	120	121	$1.0{\cdot}10^{-4}$	0.62	3.4 ms	0.05%
BSP n=120	`lc`	78,565	0.0	$1.5{\cdot}10^{-4}$	0.21	3.30 s	8.7%
PUP N=160	`la`	197	5.3	$3.8{\cdot}10^{-4}$	3.84	8.20 s	7.1%
PUP N=200	`la`	249	5.2	$5.6{\cdot}10^{-4}$	6.74	16.53 s	7.3%
HRP N=20	`la`	106	1,248	$1.3{\cdot}10^{-4}$	9.68	0.035 s	68.3%
HRP N=14	`lc`	73,143	239	$0.9{\cdot}10^{-4}$	3.44	11.09 s	99.4%

Table 4.7: Per-decision overhead on representative solved instances.

A key result is the performance gap between execution backends: compiled C++ evaluation averages $\approx 10^{-4}$ ms per consultation, whereas interpreted SWI-Prolog evaluation ranges from 0.2 to 9.7 ms.

The overall overhead varies dramatically across the benchmark configurations:

- When the heuristic has **perfect guidance (`la` on BSP)**, with only 120 decide calls taking 0.62 ms each in Prolog (and 10^{-4} ms in C++), propagator overhead accounts for just 0.05% of the total runtime.
- When the semantics gets in the way **(`lc` on BSP)**, although individual query costs remain low (0.21 ms), making 78,565 futile calls causes the propagator to consume 8.7% of runtime (17.7 s) without aiding search.
- When candidate generation becomes heavy **(`la` on HRP)**, producing over 1,200 candidates per decision increases Prolog evaluation time to 9.68 ms per call; as a result, the propagator absorbs 68.3% of the total runtime (1.03 s out of 1.55 s), even though search needs only 106 decisions.
- When frequent search and expensive evaluation collide **(`lc` on HRP)**, making 73,143 futile calls with 239 candidates each creates a worst-case scenario: Prolog evaluation and trail synchronization eat up 99.4% of the total runtime (250 s out of 264.5 s), compared to just 1.80 s on the native C++ backend.

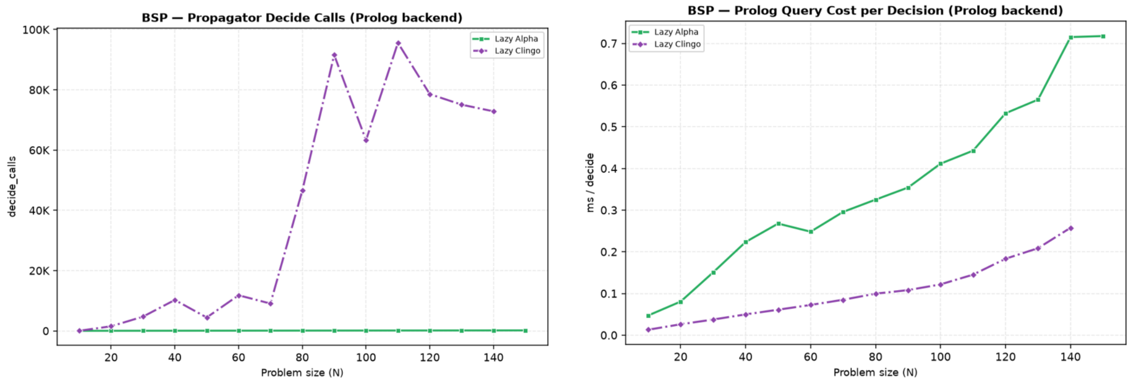


Figure 4.6: BSP, Prolog backend: decide calls and per-decision query cost. The `lc` curve pairs tens of thousands of calls with a low unit cost; `la` the opposite.

Under Alpha semantics, the query successfully identifies and evaluates all eligible candidate atoms at each step, resulting in a slightly higher query time (0.62 ms per call) due to Prolog term unification and weight calculation. Conversely, under clingo semantics, the negative body condition fails immediately on partial assignments, producing zero candidates; while this allows individual queries to terminate faster (0.21 ms), it provides no guidance whatsoever to the solver, causing a massive explosion in decision calls.

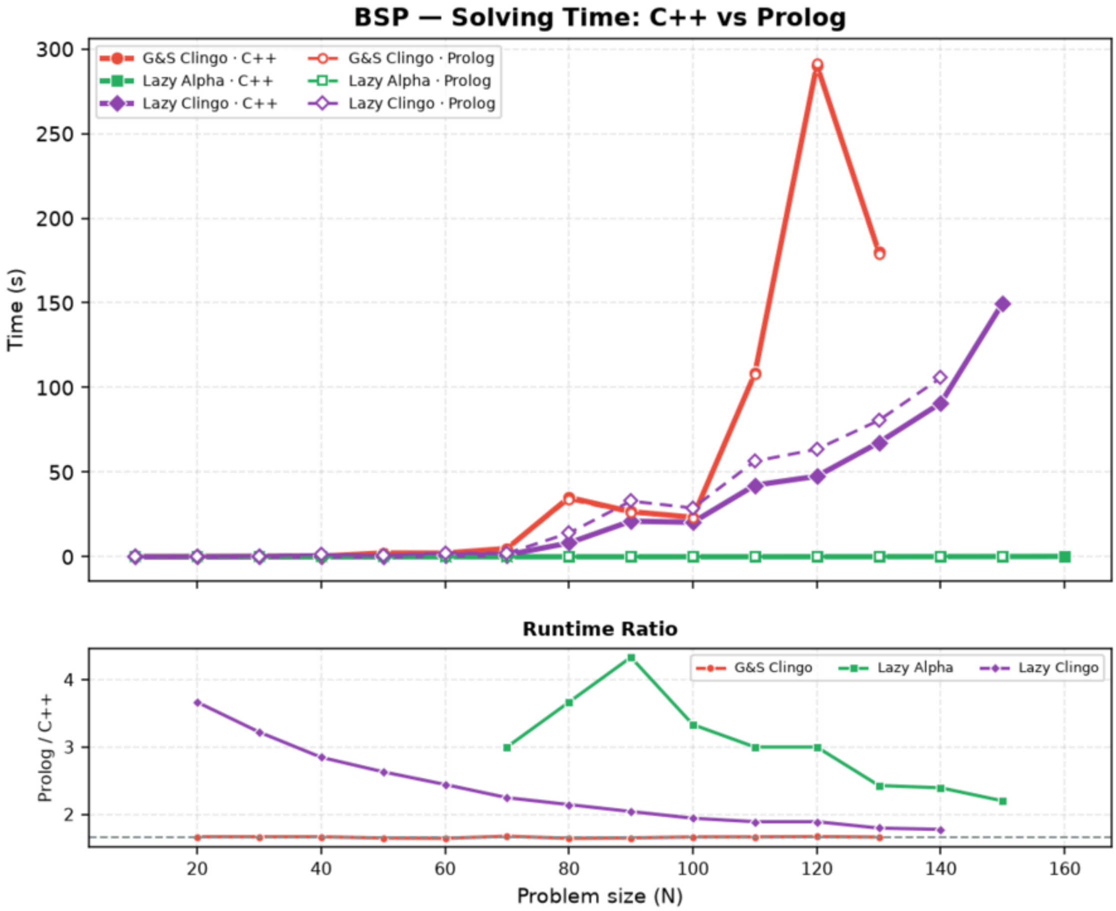


Figure 4.7: BSP: solving time comparison between the two backends. The bottom subplot shows the relative runtime ratio ($\text{Time}_{\text{Prolog}}/\text{Time}_{\text{C++}}$), where values above 1.0 indicate the factor by which the interpreted Prolog backend is slower than compiled C++.

In summary, total lazy overhead follows the relation:

$$\text{Total Overhead} = \text{Decide Calls} \times (\text{Query Cost} + \text{Amortized Synchronization Cost}).$$

Consequently, lazy evaluation is computationally cheapest precisely when the heuristic is most effective: investing additional computational effort into evaluating rich dynamic conditions per decision pays off exponentially by suppressing search space explosion.

4.7 Two Ground Simulations of Alpha: `ga` versus `ga_weak`

How closely can Alpha semantics be simulated within a ground-and-solve pipeline? Section 2.1.1 introduced two distinct transformations: `ga_weak`, which simply removes negative atoms from rule bodies, and `ga`, which replaces aggregate equality checks with dynamically unrolled monotone bounds ($S \leq \#\mathtt{sum}\{Y : c(Y)\}$).

In BSP, `ga` produces the exact same $\Theta(n^3)$ ground directives as the baseline heuristic `gc` (1,120 directives at $n = 10$, 8,440 at $n = 20$, and 27,960 at $n = 30$): keeping the equality binding instead of the unrolled bound does not reduce the grounding size, as the grounder must still enumerate all subset sums. `ga_weak`, which drops the `sum_value(S)` domain altogether, grounds only $2n$ directives (Section 3.3.1). Between the two simulations, the crucial difference lies in solver-side activation: equality bindings trigger at most one directive only after all the atoms of the sum are assigned, whereas monotone bounds activate incrementally as the partial sum increases.

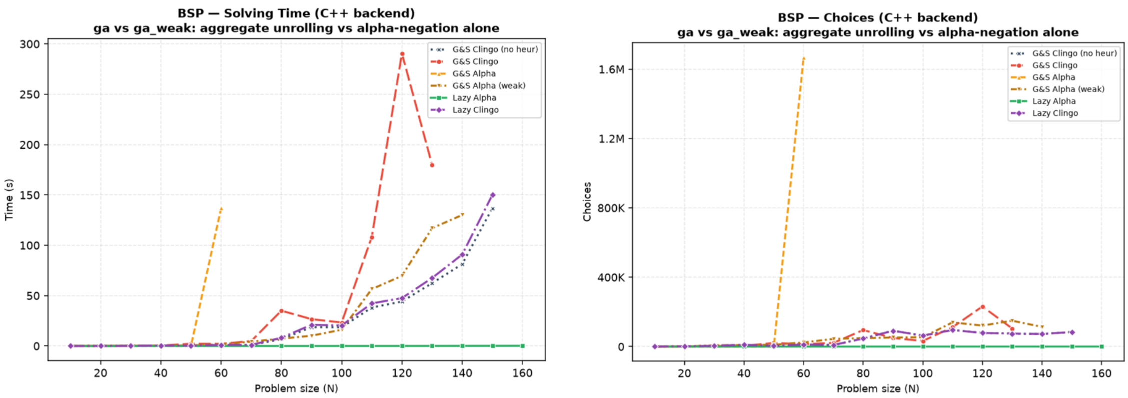


Figure 4.8: Solving time (left) and solver choices (right) for BSP comparing the α-semantics simulations (`ga` and `ga_weak`) against the baseline variants.

This illustrates the solving performance and search complexity of both simulation strategies. Beyond $n = 50$, `ga` performance rapidly deteriorates as weights exceed the `int16` range and saturate at 32,767: distinct partial sums collapse onto the same weight, and the ranking stops discriminating between candidate decisions. At $n = 60$, it already requires 1.7 million choices and 137 s of solving time, before consistently timing out from $n = 70$.

Conversely, `ga_weak` never hits the timeout, even though its weight expression is the same as `ga`'s: the equality binding is satisfied only once all the elements of the sum are assigned, so the heuristic almost never activates and the saturation issue never arises. For this same reason, however, it provides no effective guidance during search. At $n = 120$, for instance, it exhibits a degraded and noisier search trajectory, requiring 121,159 choices and 69.6 s of solving time compared to 78,565 choices and 44.1 s for the baseline.

Overall, these results show that simply omitting negative guards without unrolling aggregate bounds provides no actionable heuristic guidance to the solver, increasing overall search effort and slightly reducing the solvable instance horizon.

4.8 Evaluating Pre-Materialized Auxiliary Predicates

The `la_aux` variant investigates whether the primary advantage of lazy evaluation stems simply from reducing the number of `__heuristic` directives, or from keeping the evaluation of heuristic conditions entirely outside the ground program. In `la_aux`, heuristic conditions and aggregate values are pre-materialized into auxiliary ASP predicates (`h_b(X,S)` and `h_c(X,S)`), which the lazy propagator then queries during search.

Defining auxiliary rules forces the grounder to instantiate the full Cartesian product of elements and partial sums (X, S). As a consequence, solver variables increase to 132,756 at $n = 50$ (compared to just 5,150 in direct `la`), with peak RSS escalating to 27.2 GB at $n = 170$.

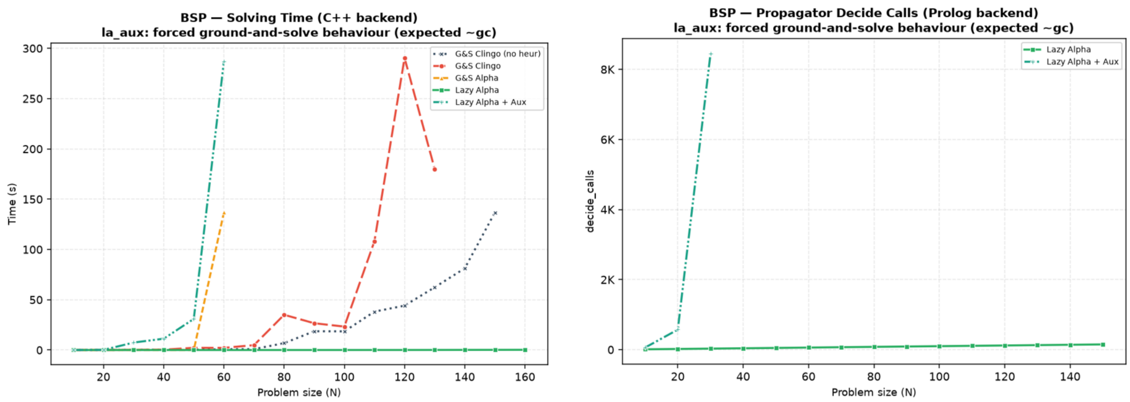


Figure 4.9: Solving time on the C++ backend (left) and number of propagator decide calls on the Prolog backend (right) comparing `la_aux` against direct `la` in BSP.

Figure 4.9 illustrates how this auxiliary materialization cripples performance across both backend architectures:

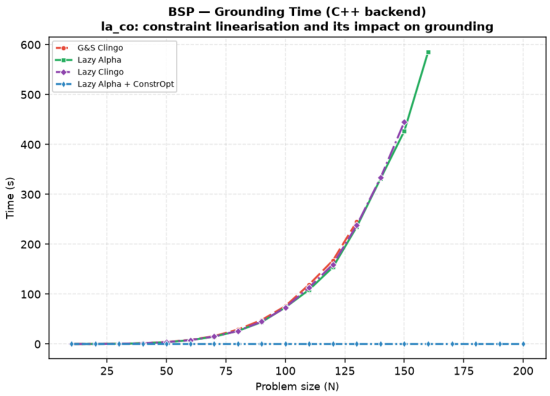
- Because the propagator must scan all materialized auxiliary candidates, the per-decision cost scales as $\mathcal{O}(n^2)$. Solving time diverges sharply from $n = 40$, reaching 287.1 s at $n = 60$ before consistently timing out from $n = 70$ onward. In contrast, the direct `la` formulation maintains negligible solving times across the entire range.
- In the Prolog backend, the sheer volume of auxiliary candidates multiplies the required consultations, jumping from 573 calls at $n = 20$ to 8,452 at $n = 30$ (compared to exactly n calls in direct `la`). Trail synchronization alone accounts for 99.2% of the total runtime (383 s out of 386 s) at $n = 30$, causing the backend to time out for all sizes $n \geq 40$.

These findings demonstrate that merely reducing the number of heuristic directives is insufficient on its own: the benefit comes from keeping the evaluation of the heuristic conditions out of the ground program. Once those conditions are materialized as auxiliary rules, the combinatorial cost simply moves from the directives to the base encoding, and the lazy propagator inherits it at every decision.

4.9 Effect of the Constraint Formulation

The `la_co` variant replaces the quadratic difference check over the two sums with a single constant-bound interval constraint, substantially simplifying the grounding structure.

In the standard formulations (`gc`, `la`, and `lc`), this quadratic constraint dominates the grounding phase: at $n = 120$, it produces approximately 52.8 million propositional rules and requires between one and three minutes of grounding time. While direct lazy evaluation (`la`) successfully eliminates heuristic directive bloat, the ground program remains constrained by the sheer size of the domain specification.



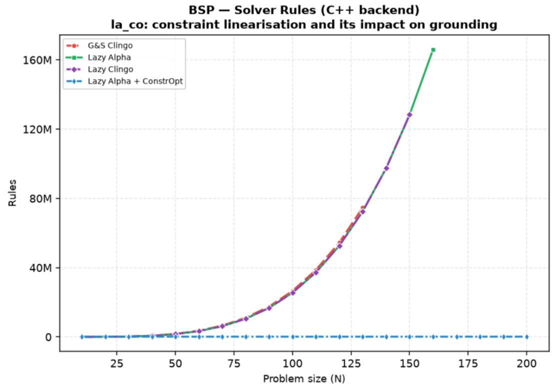


Figure 4.10: Grounding time (left) and propositional rule count (right) for BSP on the C++ backend, comparing the compact constraint formulation (`la_co`) against the baseline encodings (`gc`, `la`, and `lc`).

Figure 4.10 demonstrates the dramatic impact of this reformulation on formula size and grounding efficiency:

- While `gc`, `la`, and `lc` overlap near the top of both plots due to their shared quadratic base encoding, `la_co` flattens along the horizontal axis. At $n = 120$, the propositional formula collapses from 52.8 million rules down to just 366.
- Grounding overheads effectively disappear. Total clingo runtime at $n = 120$ drops from 154.2 s in `la` to a mere 9 ms, while fully maintaining optimal search guidance (117 choices, zero conflicts). This performance extends effortlessly up to $n = 200$, which solves in just 14 ms.

These findings yield a key architectural insight: once lazy heuristics remove the search bottleneck, the remaining bottleneck in BSP is dictated entirely by domain constraint materialization. Pairing lazy heuristics with compact, propagation-friendly encodings resolves both grounding bloat and search overhead simultaneously.

4.10 PUP: the Grounding Bottleneck at Scale

The Partner Units Problem (PUP) represents an industrial configuration benchmark where domain-specific heuristics effectively tame combinatorial search. Unlike

BSP (which relies exclusively on `#sum`), PUP combines dynamic `#count` and `#max` aggregates—notably using `#max` to track remaining capacity across adjacent partner units.

In BSP, unguided search dominated baseline execution times; in PUP, activating dynamic α-semantics reduces solving effort to just a few hundred decisions across all instance sizes. Consequently, the solving phase becomes almost trivial, shifting the performance bottleneck almost entirely ($> 95\%$ of total execution time) to the grounding phase and memory consumption. This makes PUP the ideal benchmark to evaluate the grounding and memory efficiency of lazy evaluation at scale.

Figure 4.11 and Table 4.8 summarize the overall performance across all five variants under a 600 s timeout and a 30 GB memory limit.

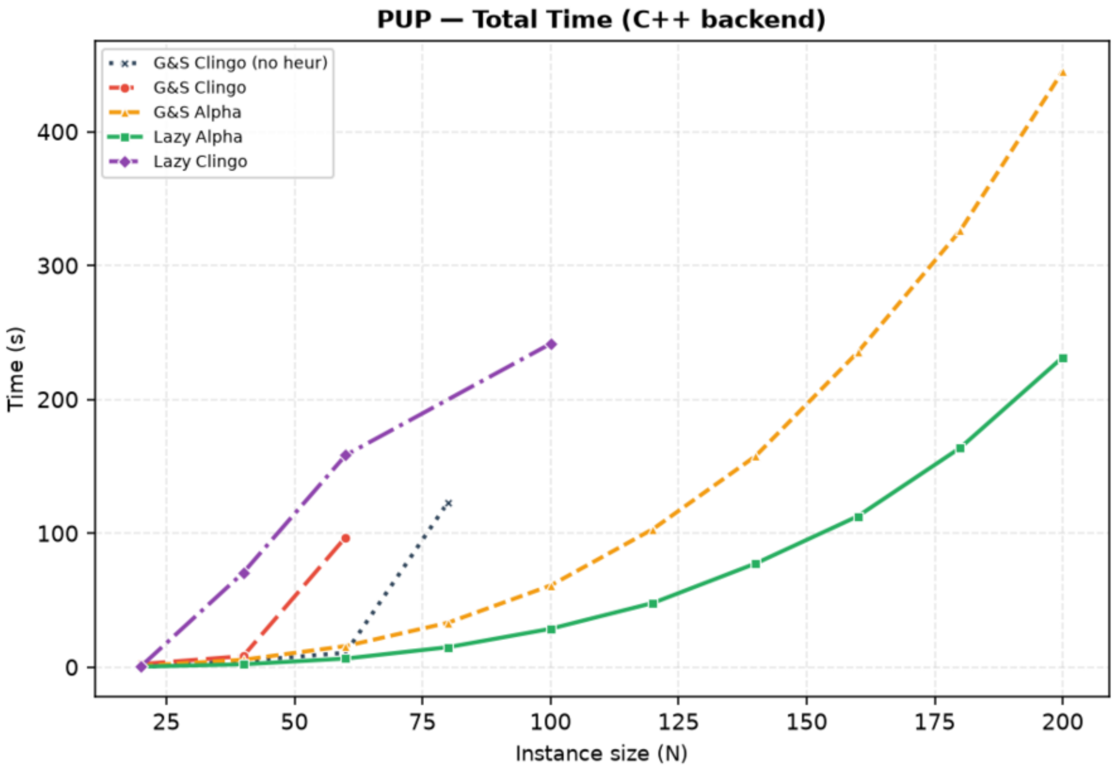


Figure 4.11: Total clingo runtime on PUP across instance sizes for the C++ backend (curves terminate at the largest solved instance).

Variant	**max N solved**	**Total (s)**	**Grounding (s)**	**Peak RSS**	**First failure**
`gc_noheur`	80	123.2	10.6	0.7 GB	T@100
`gc`	60	97.0	17.7	1.3 GB	T@80
`ga`	200	445.1	436.6	24.6 GB	—
`la`	200	231.4	225.8	12.7 GB	—
`lc`	100	241.5	27.0	2.1 GB	T@80 (non-mon.)

Table 4.8: PUP frontier on the C++ backend (600 s / 30 GB budget): largest solved `double-`N instance and corresponding resource metrics.

Analysis of baseline failures. The baseline results reveal two distinct failure dynamics:

- In `gc_noheur`, the lack of heuristic guidance forces the solver to rely entirely on VSIDS. Combinatorial choices escalate rapidly (546,549 choices at $N = 80$), leading to a timeout at $N = 100$.
- In `gc`, the standard ground heuristic performs counterintuitively worse than using no heuristic at all, failing earlier at $N = 80$ with 865,136 choices. Because clingo's static semantics requires aggregate bodies to be fully assigned before evaluating directives, the heuristic rules remain completely inactive during search. However, these inert ground directives still introduce additional variables into the solver that pollute the search space and degrade VSIDS branching decisions.

Grounding and memory savings (`la` vs `ga`). Both α-semantics variants (`ga` and `la`) solve the entire benchmark horizon up to $N = 200$, maintaining near-perfect search guidance (235 choices for `ga` vs 162 for `la` at $N = 120$). Because solving takes only a few seconds, the overall performance is differentiated almost exclusively by the grounding footprint.

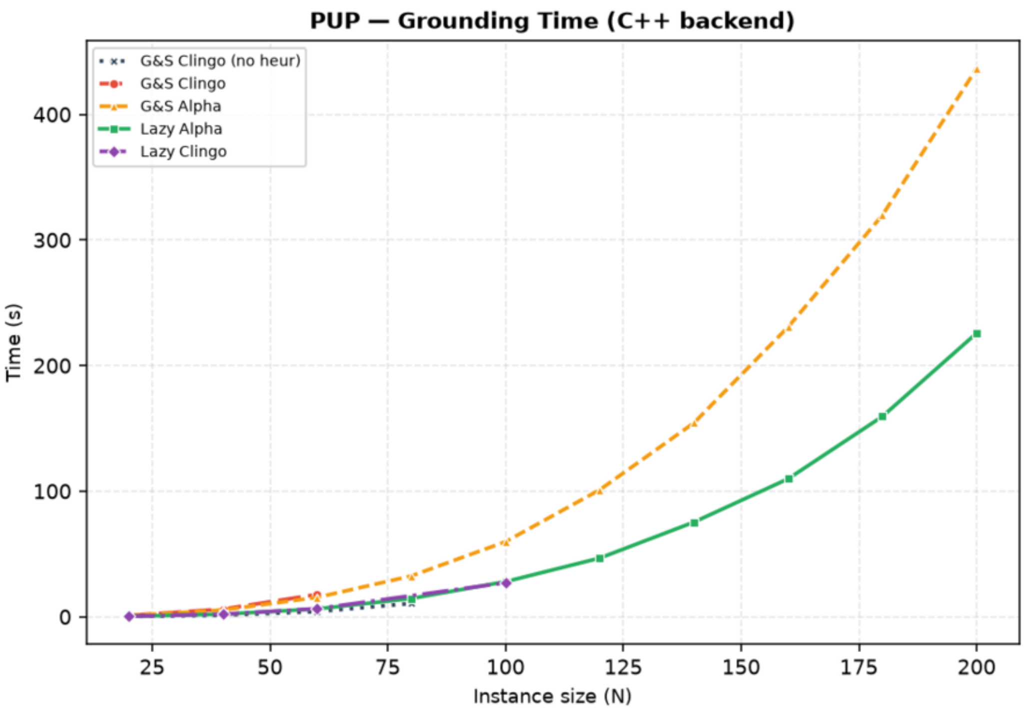


Figure 4.12: Grounding time on PUP for the C++ backend across instance sizes.

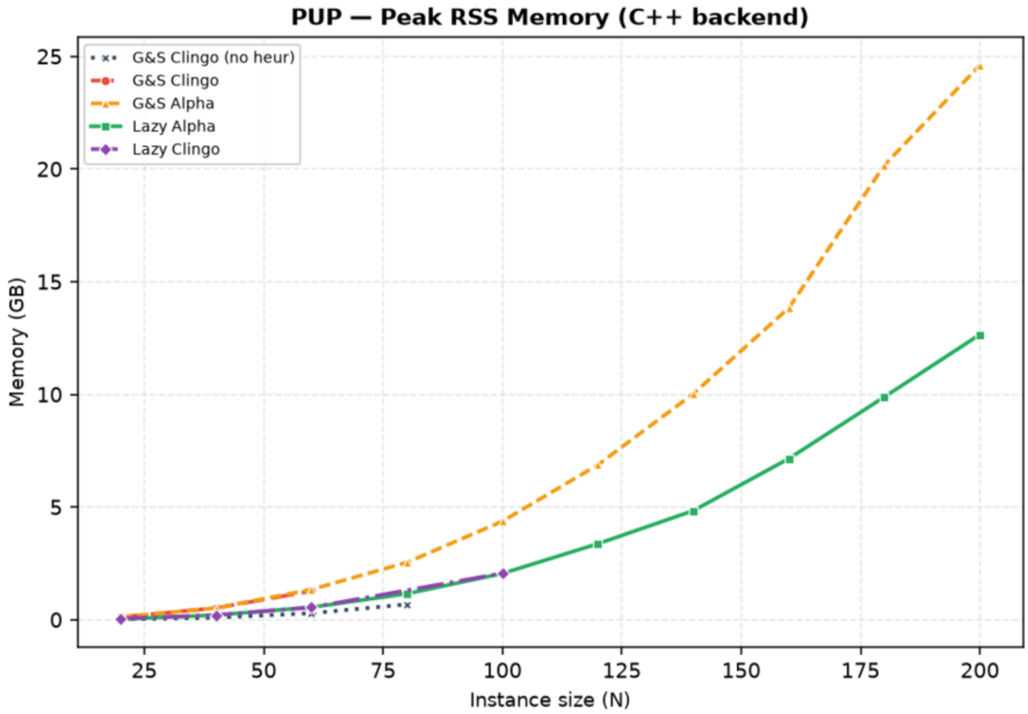


Figure 4.13: Peak memory usage on PUP for the C++ backend, illustrating the 2× memory reduction of `la` over `ga`.

As shown in Figures 4.12 and 4.13, `ga` must statically unroll all discrete bounds for the dynamic aggregates into the ground program. This unrolling incurs severe overhead, consuming 436.6 s of grounding time and 24.6 GB of memory at $N = 200$, perilously close to the 30 GB memory limit. In contrast, lazy evaluation in `la` delegates aggregate evaluation to the in-memory C++ propagator, grounding only the base problem specification. Consequently, `la` reduces grounding time to 225.8 s and peak memory to 12.7 GB, achieving a consistent 50% reduction (2× factor) in both time and memory across all tested sizes.

Controlled semantic comparison (`la` vs `lc`). In PUP, the lazy propagator requires auxiliary relations to expose aggregate sources. As a result, the ground propositional programs for `la` and `lc` are completely identical (216,193 rules at $N = 20$), differing exclusively in the runtime evaluation semantics configured within the propagator:

- `la` evaluates dynamic aggregates over currently true atoms, solving $N = 120$ in 47.9 s with only 162 choices.
- `lc` waits for complete assignments, generating zero active candidates during search. At $N = 120$, it makes 53,036 choices and times out.

The unguided nature of `lc` also explains its non-monotonic failure pattern in Table 4.8 (timing out on `double-80` but solving `double-100`). Without active heuristic guidance, the solver is subject to the high variance of CDCL restarts, occasionally finding an early solution by chance before failing definitively at larger sizes.

Both backends provide the same quality of guidance, drastically reducing the number of choices.

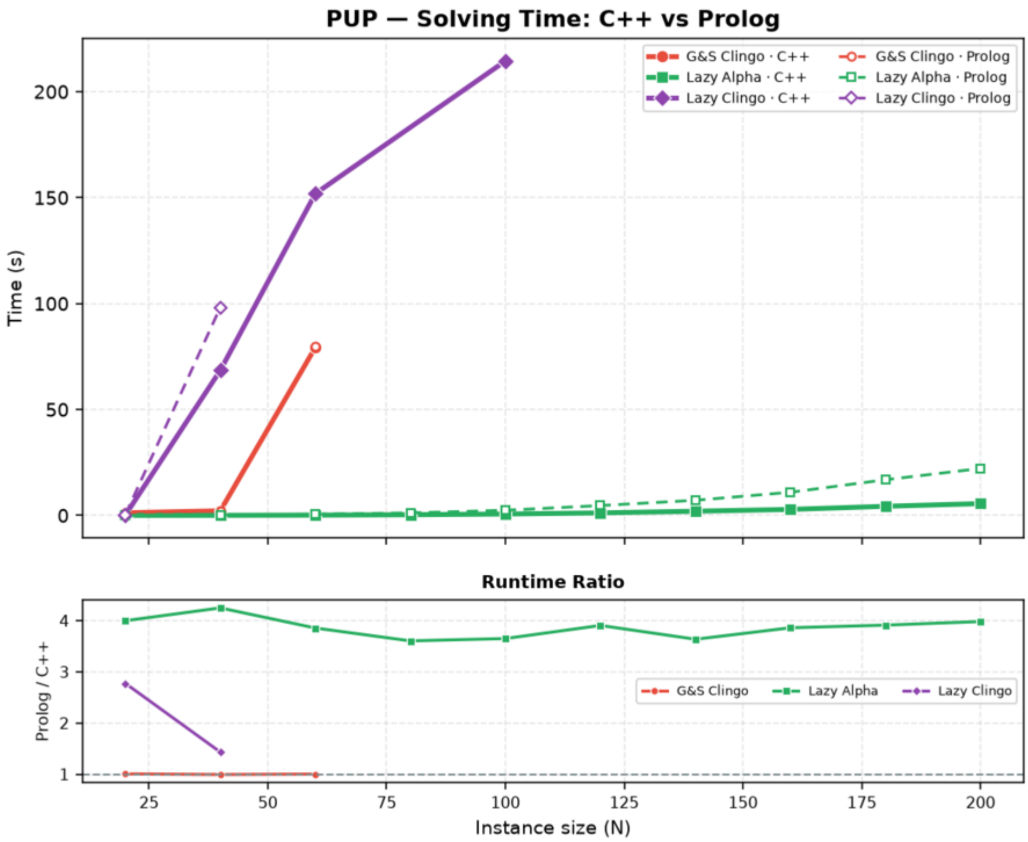


Figure 4.14: Solving time (top) and Prolog-to-C++ runtime ratio (bottom) on PUP across execution backends.

As shown in Figure 4.14, solving time remains well under 25 s across the whole horizon on both backends, with the Prolog runtime ratio remaining stable around 3.5×–4× the native C++ engine for `la`.

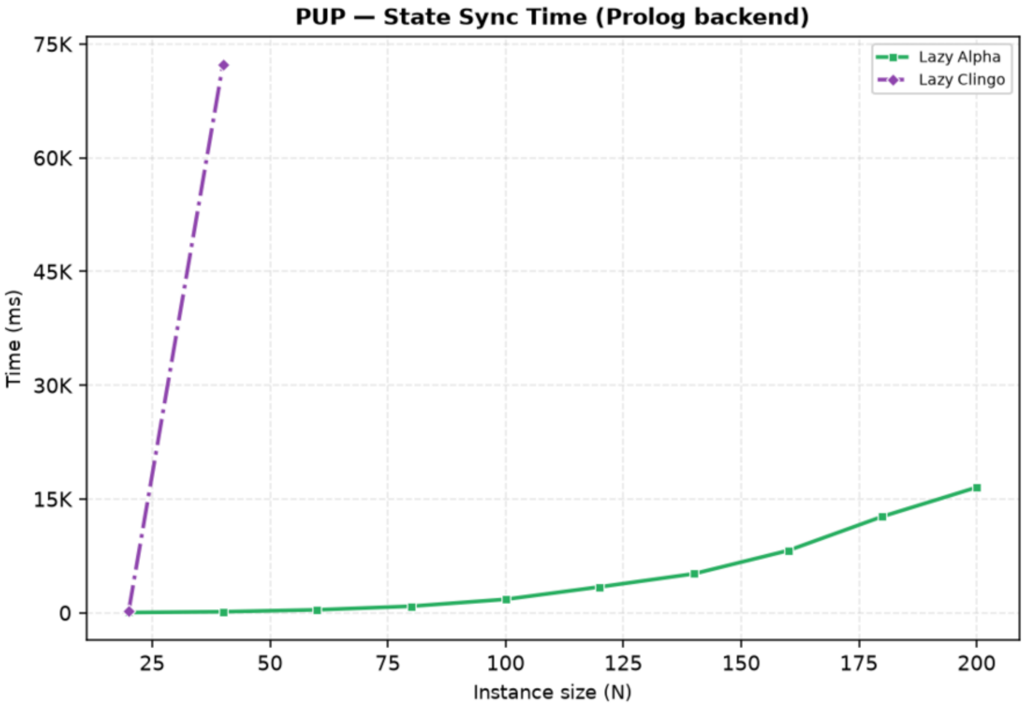


Figure 4.15: Cumulative state synchronization time for the Prolog backend on PUP.

However, as illustrated in Figure 4.15, in the Prolog backend, synchronizing solver trail accounts for 16.5 s at $N = 200$, whereas actual Prolog query evaluation takes only 1.67 s. In comparison, the native C++ backend updates its in-process aggregate structures directly along the solver trail, completing all synchronization in just 0.68 s.

4.11 HRP: Semantics Without Aggregates

HRP evaluates heuristic semantics on rules with nested default negations but without dynamic aggregates. Because HRP instances are relatively small, nearly all variants solve every instance in under a second making it an ideal testbed for guidance quality and per-decision overhead.

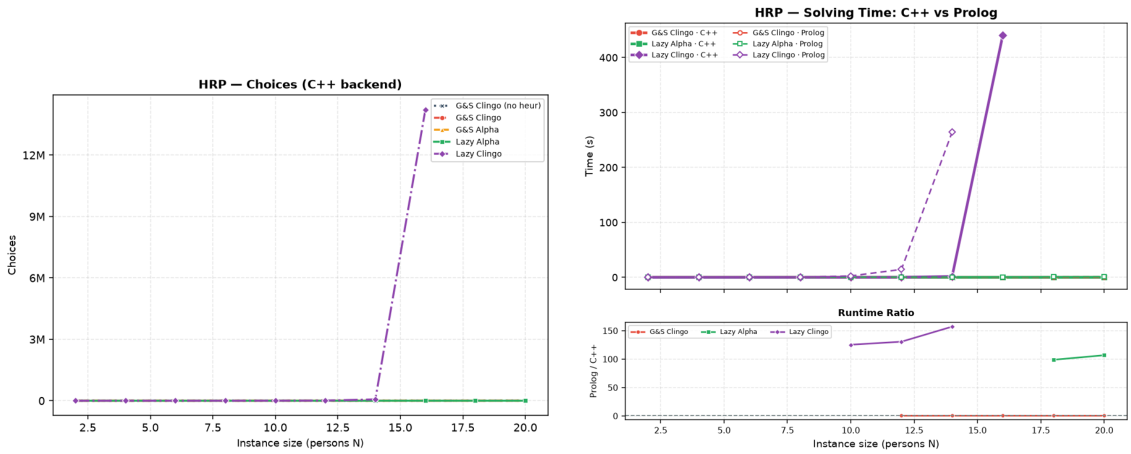


Figure 4.16: HRP: solver choices on the C++ backend (left) and the C++ versus Prolog solving comparison (right). The only diverging curve is `lc`, in both plots.

- under **Alpha-like semantics:** `la` matches `ga` choice-for-choice (86 vs 88 choices at $N = 14$, with zero conflicts for both), confirming that lazy evaluation faithfully replicates reference Alpha guidance.
- under **clingo-like semantics:** the semantics of negation delays higher-level heuristic rules while allowing incomplete lower-level rules to fire, actively misdirecting search. Consequently, `lc` requires 66,717 choices at $N = 14$ (an 800-fold increase over `la`) and 14.2 million choices at $N = 16$, timing out from $N = 18$.

Beyond guidance quality, HRP also illustrates the operational cost of relational candidate matching in the Prolog backend. Because the multi-room placement rules join multiple domain predicates, the SWI-Prolog engine must evaluate over 1,200 candidate pairs per decision at $N = 20$, driving unit query cost to 9.68 ms (Table 4.7).

For `la`, where search is resolved in only 106 choices, this per-decision effort accounts for 68.3% of the total runtime (1.03 s out of 1.55 s), compared to just 0.38 s on the compiled C++ backend. For `lc`, the combination of misdirected search and relational query overhead creates a catastrophic bottleneck: making 73,143 queries, already at $N = 14$, inflates total Prolog execution time to 264.5 s (with 99.4% spent inside the propagator), whereas the compiled native backend resolves the same misguided trajectory in 1.80 s.

4.12 An External Reference: Alpha

To evaluate how closely our propagator reproduces the lazy evaluation process and performance advantages, we compare our clingo variants against Alpha with and without its declarative heuristic engine (Qh mode).

On BSP, the guided Alpha configuration (`alpha_qh`) solves all twenty instances in 3.4 s and 0.37 GB at $n = 200$, requiring exactly n decisions and zero backtracks. This matches the search trajectory achieved by `la` in clingo (n choices, zero conflicts). The performance gap at larger sizes (585.7 s for `la` at $n = 160$ versus 2.4 s for alpha_qh) stems entirely from base program grounding in Gringo (52.8 million rules at $n = 120$), which Alpha completely avoids by instantiating rules lazily along the search trail.

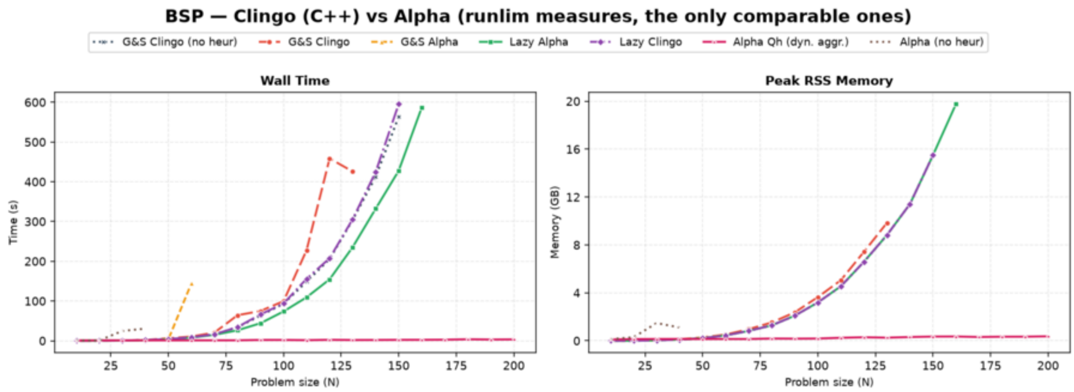


Figure 4.17: BSP benchmark dashboard: the four clingo variants and the heuristic-free baseline `gc_noheur` compared against Alpha with (`alpha_qh`, labeled *Alpha Qh*) and without (`alpha_noheur`, labeled *Alpha (no heur)*) declarative heuristics. Left: total wall-clock time; right: peak RSS. While `alpha_qh` stays flat along the horizontal axis, unguided `alpha_noheur` degrades rapidly, failing earlier than any clingo variant.

Crucially, lazy grounding alone is insufficient without dynamic heuristic guidance: `alpha_noheur` fails prematurely at $n = 40$, well before clingo's ground baseline `gc_noheur` ($n = 150$). This divergence highlights a fundamental architectural trade-off:

- **Ground-and-solve (clingo):** Because the program is fully grounded before search begins, exploring unpromising search branches is computationally very cheap: Clasp only has to perform fast boolean propagation and VSIDS variable scoring over pre-existing clauses in compiled C++.
- **Lazy grounding (Alpha):** Because rules are grounded on demand during search, exploring unpromising branches is extremely expensive: every wrong decision and subsequent backtrack forces the engine to instantiate new rules and nogoods on the fly inside the JVM runtime.

As shown in Figure 4.18, `alpha_noheur` suffers a severe search space explosion, spiking to over 67,000 guesses and backtracks at $n = 30$ before timing out at $n = 40$. Conversely, with heuristic guidance, `alpha_qh` never backtracks, materializing only the minimal subset of ground rules along the single valid solution path. This demonstrates that lazy grounding and dynamic heuristics are strictly complementary: dynamic heuristics require lazy evaluation to prevent grounding bottlenecks, while lazy grounding requires dynamic heuristics to avoid drowning in runtime instantiation overhead.

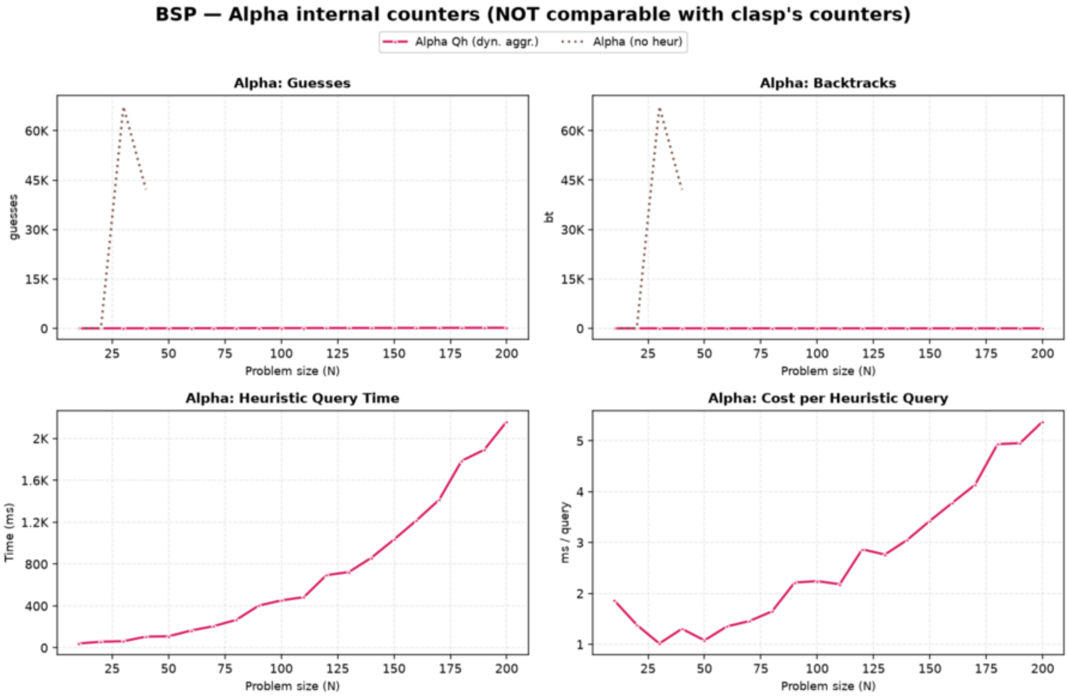


Figure 4.18: BSP internal search counters for Alpha across instance sizes. Top row: with heuristics (`alpha_qh`), guesses remain equal to n with zero backtracks; without heuristics (`alpha_noheur`), search choices explode exponentially. Bottom row: cumulative Prolog query evaluation time for `alpha_qh` (2.2 s total at $n = 200$, averaging 1.9 to 5.4 ms per consultation).

Partner Units Problem (PUP). On PUP, clingo with lazy heuristics (`la`) closely approaches Alpha's performance, solving $N = 200$ in 234.7 s and 12.7 GB compared to 50.8 s and 10.6 GB for `alpha_qh` (a factor of 4.6× in time and 1.2× in memory). In contrast, the static ground simulation `ga` requires 451.4 s and 24.6 GB (nine times the runtime and over twice the memory of `alpha_qh`). Furthermore, `alpha_noheur` solves no instances on PUP, reaffirming that dynamic heuristics are essential for solving this problem family.

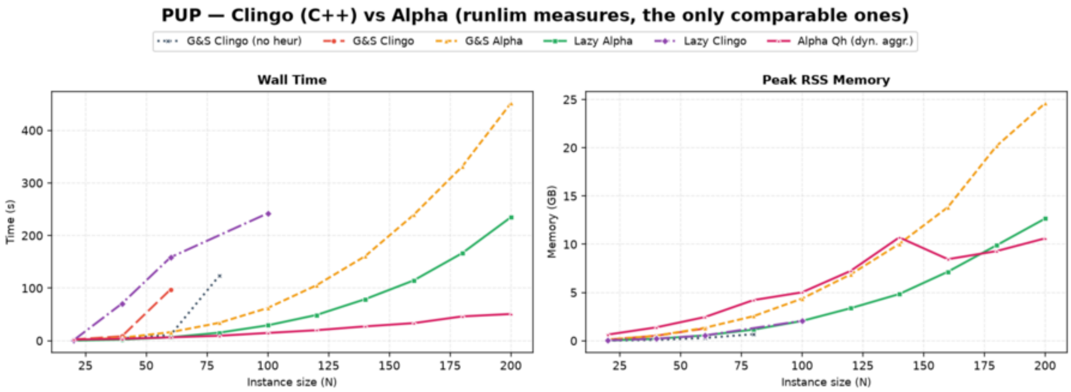


Figure 4.19: PUP benchmark dashboard: external comparison between clingo variants and Alpha. The baseline `alpha_noheur` does not appear as it solves zero instances. The lazy clingo variant `la` remains within a small factor of `alpha_qh` in both time and memory, whereas the ground simulation `ga` incurs substantial overhead on both axes.

Overall Frontier and Query Engines. Table 4.9 summarizes the maximal instance reach and resource consumption across systems. Regarding query engines, Alpha's Qh mode and the Prolog backend of this thesis both evaluate heuristics by querying an embedded Prolog engine over partial assignments. Consultation costs are of similar magnitude: at BSP $n = 120$, Alpha issues 242 queries at 2.87 ms each, compared to 120 consultations at 0.62 ms for our Prolog backend. This confirms that the synchronization-based propagator design achieves competitive performance relative to reference logic programming engines.

Family	**System**	**max solved**	**Wall (s)**	**Peak RSS**
BSP	`alpha_noheur`	40	30.5	1.1 GB
BSP	clingo `gc_noheur`	150	563.6	15.5 GB
BSP	clingo `la`	160	586.5	19.8 GB
BSP	`alpha_qh`	200	3.4	0.37 GB
BSP	clingo `la_co`	200	0.04	< 0.1 GB
PUP	`alpha_noheur`	—	—	—
PUP	clingo `gc_noheur`	80	123.3	0.7 GB
PUP	clingo `lc`	100	242.2	2.1 GB
PUP	clingo `ga`	200	451.4	24.6 GB
PUP	clingo `la`	200	234.7	12.7 GB
PUP	`alpha_qh`	200	50.8	10.6 GB

Table 4.9: Largest instance solved within the 600 s / 30 GB budget, and its cost, ordered by reach within each family. Wall time and peak RSS are `runlim` external process measures; Alpha's RSS includes the reserved JVM heap. clingo rows represent the native C++ backend. A dash marks a configuration that solved no instances.

4.13 C++ versus Prolog: Backend Comparison

Across the benchmark campaign, the compiled C++ and interpreted Prolog backends demonstrate consistent behavior, producing identical solvability frontiers in all but five specific configurations. Whenever heuristic consultations are infrequent or computationally lightweight—such as in `la` on PUP and HRP—both engines achieve the exact same problem reach.

Where the two backends diverge, the Prolog engine consistently fails earlier due to evaluation and synchronization overhead. On BSP, the frontier drops by a single step for `la` ($n = 160 \rightarrow 150$) and `lc` ($150 \rightarrow 140$) because both runs finish within seconds of the 600-second time limit. On PUP ($N = 100 \rightarrow 40$) and HRP ($N = 16 \rightarrow 14$) under `lc`, the unguided search triggers tens of thousands of futile queries, turning Prolog's per-call interpretation cost into the dominant bottleneck.

The C++ backend evaluates heuristic decisions three to four orders of magnitude faster than the interpreted Prolog engine ($\approx 10^{-4}$ ms versus 0.2–9.7 ms per call); hence, the native backend provides maximum raw performance for solving, while the embedded Prolog backend offers expressive relational querying and transparent rapid prototyping.

4.14 Summary of Results

The empirical evaluation demonstrates that lazy heuristic evaluation fundamentally changes how domain-specific guidance interacts with ground-and-solve ASP. By shifting dynamic aggregate evaluation from grounding to search, the framework reduces the heuristic representation from a combinatorial $\Theta(n^3)$ explosion down to two constant template facts on BSP. In realistic configuration benchmarks like PUP, this translates into halving both grounding time (225.8 s versus 436.6 s) and peak memory (12.7 GB versus 24.6 GB) at scale compared to static ground simulation (`ga`). Furthermore, the exploratory variants confirm that this benefit requires keeping the evaluation entirely dynamic: omitting negative guards without unrolling dynamic aggregates (`ga_weak`) fails to provide actionable search guidance, while pre-materializing heuristic bodies into auxiliary ASP rules (`la_aux`) reintroduces the combinatorial explosion into the base program. Conversely, pairing lazy heuristics with compact domain constraints (`la_co`) eliminates grounding and solving bottlenecks simultaneously, resolving BSP up to $n = 200$ in milliseconds.

The campaign also proves that evaluation semantics is decisive for search guidance. Under Alpha-like semantics, dynamic aggregate evaluation provides immediate, actionable advice on partial assignments, achieving near-perfect search on BSP (n choices with zero conflicts) and faithful multi-level ranking on HRP. In contrast, standard clingo semantics requires complete assignments to evaluate aggregate bodies, rendering the heuristic completely inert on BSP and actively misleading on HRP, where delayed high-level rules allow incomplete lower-level rules to steer the solver into massive search subspaces. Moreover, while static ground unrolling (`ga`) can approximate dynamic aggregates on small instances, it slows down grounding and breaks down whenever the flattened weights overflow Clasp's 16-bit ceiling (32,767), a limitation that the lazy propagator avoids entirely by sorting decisions internally.

Finally, lazy evaluation trades grounding memory for decision-time computation, where total overhead factorizes into decision frequency multiplied by per-consultation cost. As a result, the lazy mechanism is computationally cheapest precisely when the heuristic guides well, as the solver makes only a minimal number of decisions. Compared against the reference lazy solver Alpha, our clingo propagator reproduces the exact reference search trajectories, coming within a factor of 4.6× in execution time and matching peak memory on PUP while avoiding the static unrolling overhead of standard ground-and-solve systems.

Chapter 5

Discussion and Conclusions

5.1 Discussion of the Results

The work shows that an important part of the cost of declarative dynamic heuristics can be moved from grounding to search. On BSP, native `#heuristic` directives grow as $\Theta(n^3)$ and exceed one million ground objects at $n = 100$; the lazy variants keep two facts, and the difference is visible in the propositional instance size, in the peak memory, and in the timeout profile. The mechanism generalizes, and the full campaign measures the generalization: on PUP, whose heuristics multiply four unrolled value domains per ground pair, the ground Alpha simulation and the lazy variant both clear the entire tested range under a raised 600-second/30 GB budget, but the ground simulation costs twice the time and twice the memory of the lazy variant at every one of the ten common sizes, because the unrolled directives keep growing with the value domains being unrolled; both dominate the unguided baseline, which stalls by timeout at less than half their reach. Where the heuristic grounding is the binding constraint, the lazy representation converts directly into a large cost advantage over the best ground heuristic, and into solved instances relative to the unguided baseline; where the base program dominates, as at the top of the BSP range, it converts into an unchanged frontier reached with the search component reduced by three orders of magnitude, and the residual wall belongs to the domain encoding, not to the heuristic. The BSP frontier is unchanged rather than improved: the lazy variant, the clingo-like one and the heuristic-free baseline all stop within two sizes of each other, and the runs that decide the exact size finish within a few percent of the time limit. On that family the claim is about cost rather than reach, and the campaign's design — structural counters alongside wall-clock times — is what keeps the two apart.

The external comparison of Section 4.12 puts a number on both halves of that sentence, and it is the most informative single experiment of the campaign because it is the only one whose yardstick is not clingo. Against Alpha in its dynamic-aggregate variant — the system whose semantics this thesis reproduces, running the authors' own encodings — the lazy propagator delivers the guidance and not the grounding. On BSP it reproduces the reference trajectory exactly, n decisions and no backtracking, and then loses two orders of magnitude in wall time to a system that never builds the 52.8 million rules of the base program. On PUP, where the base program is proportionally less dominant, it lands within a factor of 4.6 in time and 1.2× the peak memory, while the ground simulation of the very same heuristic is nine times slower and more than twice as hungry as the reference. Read as a whole the ordering is `ga` < `la` < Alpha, with the lazy representation recovering most of the distance and grounding accounting for the remainder, which is the claim the thesis makes, now measured against an external reference rather than asserted.

The same experiment also protects the semantic claim from an obvious objection, namely that the Alpha reading only looks good because it is being compared with clingo baselines. Alpha *without* its declarative heuristic stops at $n = 40$ on BSP, eleven sizes before ground clingo without a heuristic, and solves no PUP instance at all. Lazy grounding and dynamic heuristics are therefore separable contributions and neither is sufficient on its own: the four combinations of (eager, lazy) grounding and (absent, dynamic) heuristic land at $n = 150$, $n = 160$, $n = 40$ and $n = 200$ respectively. The effect this thesis isolates inside clingo is the same effect that makes the reference system usable.

This result should not be read as a replacement for general lazy grounding. clingo still grounds the base program; the propagator acts only on the heuristic component, and Section 4.12 measures exactly what that leaves on the table. Nor should it be read as a claim that laziness is free: the runtime work is real, it is paid at every decision, and the per-decision metrics exist precisely to keep that cost on the record. The honest formulation is a space–time trade-off with a crossover: below it, ground-and-solve remains competitive, because the explosion is manageable and the lazy machinery (in the Prolog case, an entire in-process logic-programming engine) carries a fixed overhead; beyond it, the constant-size heuristic component is the only representation that survives.

The second central finding is semantic, and it is a finding about *meaning*, not performance. The same declarative heuristic changes behaviour dramatically depending on how partial information is read. Under the Alpha-like reading, the BSP heuristic is a perfect greedy strategy (n choices, zero conflicts) and the HRP policy reproduces its intended level structure decision for decision. Under the clingo-like reading, the very same weights and bodies degenerate in two distinct ways that the campaign now documents separately: on BSP the guards never open at decision time, the propagator reports zero applicable candidates per call, and the search is bit-identical to the baseline (inertness); on HRP the surviving low-level modifications actively steer the solver into a region almost eight hundred times larger at $N = 14$ and five orders of magnitude larger at $N = 16$, with both backends independently reaching the same pathological order of magnitude of choices even though their exact counts differ (misguidance). Neither reading is "wrong": they answer different questions, and the four-variant matrix exists to keep those questions separate. A comparison that silently mixed the axes, for example by measuring `la` against `gc` and attributing the difference to laziness, would be methodologically invalid; the thesis takes some care never to do so. The campaign adds a corollary about the ground side of the matrix. The faithful ground simulation of the Alpha reading, `ga`, reproduces the lazy guidance exactly while its weights remain representable, and stops working when they do not: on BSP that happens at $n = 51$, and from $n = 70$ the variant no longer finishes at all, making it the earliest-failing variant of the main matrix on BSP, nine sizes before the baseline. The limit is not the cost of the unrolling, which on that family adds two tenths of a second of grounding and thirty-seven megabytes, but clingo's weight field, into which the reconstructed value of the aggregate has to fit. Inside a ground-and-solve pipeline the lazy propagator is therefore not merely the cheaper carrier of the Alpha semantics; beyond a certain instance size it is the only one that can carry it at all, because it ranks its targets internally and never has to encode the aggregate as a weight.

The third finding concerns the translation layer between the two heuristic languages. Alpha's weight–priority pairs are global levels; clingo's priorities arbitrate on a single atom. Every encoding that expresses an ordering between different atom families must therefore flatten its levels into weights wherever clingo's reading applies: in the ground variants, because clasp is the consumer, and in the lazy clingo variants, because the propagator deliberately reproduces clingo's behaviour there. This is easy to state and easy to get wrong: an early revision of the lazy-clingo PUP and HRP encodings carried Alpha-style priorities that the clingo semantics does not rank globally, producing an unintended decision order that inverted the designed one (sensors before zones; cabinet-filling before reuse), and diverging from the Prolog twin encodings of the same variants. The final audit caught and corrected the discrepancy by applying the same flattening used in the ground encodings. The episode is instructive: the interaction between priority semantics and evaluation backend is exactly the kind of silent, non-crashing behavioural divergence that a benchmark pipeline cannot detect from exit codes, and it motivates the equivalence and wiring guards that the suite now includes.

The lesson was then turned into a structural decision rather than a matter of vigilance. As long as the priority field is used to carry levels in *some* encodings, its meaning depends on the pair (semantics, backend): a global level under one reading, a purely local arbiter under another, and a global level again in the Prolog backend, whose selection rule sorts by (priority, weight) regardless of the semantics selector. Three readings of one syntactic field is a standing invitation to the very bug just described. Since flattening is order-preserving, the suite removes the ambiguity by construction: the levels always live in the weight, in every variant and in both backends. The native heuristic language went one step further and dropped the priority construct altogether, so that a directive written for it cannot express a level except through its weight, and the parser rejects the field rather than silently reinterpreting it. Two properties follow. First, the encodings of a pair differ by exactly one token, so the axis under study is the only thing that varies. Second, the change is behaviour-preserving with respect to the earlier revision: on every benchmark the flattened weights reproduce the previous total order exactly, because on each instance the level multiplier strictly dominates the intra-level weights, so the experimental results reported here are unaffected.

A related engineering lesson comes from the silent-fallback failure mode of the Prolog backend. A single missing environment variable degrades every lazy Prolog run to the no-heuristic baseline without any error, since the fallback evaluator simply fails on the Prolog-style rule bodies. The pipeline defends itself with positive evidence of activity (`decide_calls > 0`), cross-backend agreement checks, and a wrapper that forces the variable; the general principle, that a heuristic mechanism must prove it ran rather than merely not fail, seems worth stating explicitly.

Finally, the auxiliary-form experiment delimits the approach from within. `la_aux` shows that laziness only pays if the *entire* dynamic part of the heuristic, aggregate included, stays out of the grounder: materializing the heuristic body into an auxiliary relation reintroduces the very product the mechanism was built to avoid.

A fourth lesson concerns the interface between the heuristic language and the solver that consumes it. clingo represents the weight of a `#heuristic` modification in a bounded field and saturates anything larger, so a weight is not an arbitrary integer but a resource with a ceiling. Any encoding that packs information into the weight, whether an Alpha level through the flattening rule of Section 2.1.3 or the value of an aggregate through the unrolling of `ga`, is spending that resource, and the two uses compete. Nothing warns the modeller when the ceiling is reached: the program still grounds, the run still terminates, the answer sets are still correct, and only the search degrades. This is the same class of silent failure as the priority episode and the Prolog fallback described above, and it argues for the same remedy, namely that a heuristic layer should check its own preconditions rather than trust that they hold. The lazy propagator sidesteps the ceiling by construction, since it ranks targets internally and hands the solver a decision rather than a weight, and that is a design advantage worth stating separately from the grounding one.

5.2 Limitations

The current implementation has explicit technical limitations. The native backend indexes targets and bodies through numeric tuples only; non-numeric arguments are not supported. The arithmetic language covers addition, subtraction, and multiplication; the available dynamic aggregates are sum, count, min, and max, over a single source predicate, with joins delegated to precompiled auxiliary predicates in the encoding. The modifiers `init` and `factor` are recognized but deliberately rejected: in native clingo they modify solver-internal scores that the `decide` interface cannot reach, and approximating them as `level` would silently change their meaning. The propagator is registered sequentially and has no synchronization layer for parallel solving. The tie-break between targets of equal rank is deterministic but does not reproduce clingo's internal scores.

On the evaluation side, the campaign covers all three families on both backends with the corrected encodings, and both backends are phase-timed, but its scope needs stating precisely. Every cell is a single run, so the campaign measures no within-cell variance directly. What it does measure is reproducibility across machines: the 46 pairs of runs that execute byte-identical ground programs on different nodes agree on grounding time to a median of 2.2% and a maximum of 6.6% (Section 4.1.1). Time-based claims are therefore reported as measurements, and the separations this thesis rests on — the factor of two between `ga` and `la` on PUP, the two orders of magnitude of `lc` on HRP, the three orders of magnitude on the BSP solving component — sit far above that figure. The exception is the exact position of the BSP frontier, where the deciding runs finish within 2.5% of the time limit; repeated runs per cell would settle those two sizes, and remain the most valuable extension of the protocol.

The HRP instances remain easy for clasp at every benchmarked size, so HRP supports conclusions about guidance quality and per-decision cost, not about frontiers. The PUP frontier claims rest on one instance per size (the `double` family): the non-monotone behaviour observed for `gc` and `lc` is a reminder that neighbouring sizes differ in hardness, and per-size variance is not measured. The two backends also do not reproduce identical search trajectories on PUP and HRP, where the heuristic ranking admits ties that the C++ and Prolog tie-breaks resolve differently; the divergence is a few percent on `la` and larger on the unstable `lc` searches, which is why the cross-backend comparisons in those families are read qualitatively rather than as controlled measurements of the engine alone. The memory metric is end-to-end process RSS, which is the honest but blunt instrument discussed in Chapters 3 and 4: it includes the Prolog engine for the lazy Prolog variants and cannot separate grounding memory from search memory. The auxiliary clingo evaluator of the Prolog build re-grounds a program per decision and is a functional reference, not a performance-comparable backend. The external comparison with Alpha inherits limitations of its own: it covers BSP and PUP but not HRP, whose reference heuristic contains no dynamic aggregates and would therefore measure a different thing; the two solvers' internal counters are not commensurable, so the comparison rests on the two `runlim` measures alone; and Alpha's peak RSS includes the JVM heap reserved through `-Xmx` rather than the heap in use, which makes its memory figures an upper bound and the memory comparison on PUP a tie read conservatively rather than a measured equality. Finally, the whitelist of static predicates and the inference of the program constant n in the Prolog backend are benchmark-specific conveniences that a production version should replace with a declarative interface.

5.3 Implications

The main implication is that declarative heuristics can be treated as an intermediate layer between modelling and solving. One does not have to choose between fully grounded `#heuristic` directives and procedural heuristics hard-coded in the solver: a compact declarative surface, evaluated at runtime with access to the partial assignment, is a workable third point in the design space, and it can host more than one evaluation semantics behind one syntax. The two backends demonstrate two implementation strategies for that layer, a compiled, incremental one and an embedded general-purpose engine, with the expected trade-off between per-decision efficiency and expressive freedom of the rule bodies.

From an engineering perspective, the work also shows that clingo's propagator and heuristic interfaces are flexible enough to prototype non-trivial guidance mechanisms without touching the solver core: watched literals, incremental aggregate states, per-target modifier resolution, and a global decision ranking are all expressible against the public API of `clingo::Heuristic`.

5.4 Future Work

Several extensions follow naturally. The mechanism should become configurable rather than always-on, and the expression language deserves integer division, modulo, and comparisons, together with a principled treatment of non-numeric symbols through a stable internal mapping. Aggregates with in-template joins would remove the need for precompiled auxiliary predicates. On the Prolog side, the synchronization protocol could move from per-literal goals to batched updates, and the backend could expose its per-decision statistics through clingo's statistics interface instead of stderr parsing. A parallel-solving synchronization layer and a tie-break policy that cooperates with the solver's own scores are natural solver-side improvements.

On the evaluation side, the PUP synchronization numbers make the batched-update optimization concrete and measurable: at `double-160` the trail mirroring costs eleven times the queries it serves (8.2 seconds against 0.75), and at `double-200` ten times (16.5 against 1.7), while the native backend performs the same task in 0.68 seconds; a protocol that coalesces retract–assert pairs per propagation batch therefore has both a quantified target to beat and an existence proof that the target is reachable. Beyond that, configuration, scheduling, and packing domains are natural candidates for broadening the evaluation, precisely because their useful heuristics are dynamic; PUP would additionally benefit from multiple instances per size (the `doubleV` family) to put variance bars behind the frontier claims. A further direction is a verified converter from annotated `#heuristic` directives to lazy facts, with the flattening rule of Section 2.1.3 applied mechanically rather than by hand; the audit episode discussed above is a concrete argument for automating that step.

5.5 Conclusions

This thesis presented a lazy evaluation mechanism for declarative domain-specific heuristics in clingo, instantiated in two backends over a common propagator design: a native C++ backend interpreting compact `__heuristic` facts with incremental dynamic aggregates, and an in-process SWI-Prolog backend evaluating heuristic rules as logic programs against a synchronized image of the solver trail. The mechanism supports two explicit readings of partial information, the clingo-like and the Alpha-like semantics, and reproduces clingo's weight, priority, and modifier behaviour where that reading applies, including the systematic flattening of Alpha's global priority levels into clingo weights.

The full campaign on BSP, PUP, and HRP under uniform limits (600 seconds, 30 GB) shows a reduction of the ground heuristic component from $\Theta(n^3)$ directives, more than one million at $n = 100$, to two constant facts, with the corresponding saving in real memory wherever the heuristic grounding matters; on PUP the lazy variant reaches more than twice the solvable frontier of the unguided baseline and costs half the time and half the memory of the best ground heuristic at every size both solve, while on BSP, where the base program owns the frontier, it removes the search cost outright — n choices and zero conflicts at every size, against tens of thousands for every alternative — and reaches the same wall as the baseline having spent 0.04 seconds of solving where the baseline spends 44. The campaign also quantifies the price, on both backends: a per-decision runtime cost that ground-and-solve does not pay, which factorizes into consultation count times consultation cost, ranges from nothing to 99% of a run, differs between the compiled and the interpreted evaluator by three to four orders of magnitude per consultation, vanishes when the heuristic guides well, and dominates when it does not. The suite validates that all variants and both backends preserve the answer sets and guards against silent misconfiguration. Measured against Alpha itself, running the authors' encodings, the lazy propagator reproduces the reference guidance and closes most of the distance to a native lazy-grounding solver — within a factor of 4.6 in time and $1.2\times$ its memory on PUP, where a ground simulation of the same heuristic is instead nine times slower than the reference — with the residual gap consisting entirely of the base program that clingo still grounds.

The final result is a working, documented, and measurable prototype together with a methodology: two orthogonal axes that keep representation and semantics separate, a flattening rule that makes Alpha-style encodings meaningful under clingo's reading of priorities, and an experimental discipline that states what each number measures. It does not solve the grounding bottleneck in general, but it provides a concrete, verifiable path for bringing dynamic declarative heuristics into clingo without always paying the cost of their full ground materialization.

Bibliography

[1] Michael Gelfond and Vladimir Lifschitz. "The Stable Model Semantics for Logic Programming". In: *Proceedings of the Fifth International Conference and Symposium on Logic Programming*. MIT Press, 1988, pp. 1070–1080.

[2] Torsten Schaub. *Answer Set Solving in Practice*. Potassco slide package, University of Potsdam. Version 1.22.0, 4 October 2024. 2024. URL: `https://teaching.potassco.org`.

[3] Martin Gebser et al. *Potassco Guide*. Second edition, version 2.2.0. University of Potsdam. 2019. URL: `https://potassco.org`.

[4] Francesco Calimeri et al. "ASP-Core-2 Input Language Format". In: *Theory and Practice of Logic Programming* 20.2 (2020), pp. 294–309.

[5] Martin Gebser et al. "Domain-Specific Heuristics in Answer Set Programming". In: *Proceedings of the Twenty-Seventh AAAI Conference on Artificial Intelligence*. 2013, pp. 350–356.

[6] Carmine Dodaro et al. "Combining Answer Set Programming and Domain Heuristics for Solving Hard Industrial Problems". In: *Theory and Practice of Logic Programming* 16.5-6 (2016), pp. 653–669.

[7] Antonius Weinzierl. "Blending Lazy-Grounding and CDNL Search for Answer-Set Solving". In: *Logic Programming and Nonmonotonic Reasoning (LPNMR 2017)*. Vol. 10377. Lecture Notes in Computer Science. Springer, 2017, pp. 191–204.

[8] Richard Comploi-Taupe et al. "Domain-Specific Heuristics in Answer Set Programming: A Declarative Non-Monotonic Approach". In: *Journal of Artificial Intelligence Research* 76 (2023), pp. 59–114.

[9] Richard Comploi-Taupe, Gerhard Friedrich, and Tilman Niestroj. "Dynamic Aggregates in Expressive ASP Heuristics for Configuration Problems". In: *Proceedings of the 25th International Workshop on Configuration (ConfWS 2023)*. Vol. 3509. CEUR Workshop Proceedings. CEUR-WS.org, 2023.

[10] Martin Gebser et al. "Multi-shot ASP solving with clingo". In: *Theory and Practice of Logic Programming* 19.1 (2019), pp. 27–82.

[11] Martin Gebser et al. "clasp: A Conflict-Driven Answer Set Solver". In: *Logic Programming and Nonmonotonic Reasoning (LPNMR 2007)*. Vol. 4483. Lecture Notes in Computer Science. Springer, 2007, pp. 260–265.

[12] Matthew W. Moskewicz et al. "Chaff: Engineering an Efficient SAT Solver". In: *Proceedings of the 38th Design Automation Conference*. 2001.

[13] Jan Wielemaker et al. “SWI-Prolog”. In: *Theory and Practice of Logic Programming* 12.1-2 (2012), pp. 67–96.

[14] Markus Aschinger et al. “Optimization Methods for the Partner Units Problem”. In: *Integration of AI and OR Techniques in Constraint Programming (CPAIOR 2011)*. Vol. 6697. Lecture Notes in Computer Science. Springer, 2011, pp. 4–19.

[15] Gerhard Friedrich et al. “(Re)configuration using Answer Set Programming”. In: *Proceedings of the IJCAI 2011 Workshop on Configuration*. Vol. 755. CEUR Workshop Proceedings. CEUR-WS.org, 2011, pp. 17–24.

[16] Potassco Group. *benchmark-tool: A tool for benchmarking solvers and orchestrating cluster runs*. `https://potassco.org/benchmark-tool/`. Accessed 2026.

[17] Armin Biere. *runlim: a tool for sampling time and memory usage of a process*. `https://github.com/arminbiere/runlim`. Accessed 2026.

Acknowledgements

I would like to thank Prof. Carmine Dodaro for his guidance at the University of Calabria, and Prof. Torsten Schaub and Dr. Javier Romero for their supervision and support during my Erasmus period at the University of Potsdam. Their feedback helped shape both the modelling choices and the implementation decisions behind this work.

I am also grateful to the ASP and Potassco communities. Tools such as clingo make it possible to experiment with advanced research ideas while relying on a solid, well documented, and open foundation.

Finally, I thank my family and the people who stayed close to me throughout this thesis work, for their constant support and patience during the many long debugging sessions.